claude-windows / c6028e72-ebfd-4218-89a8-c4d7dd23be96

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c6028e72-ebfd-4218-89a8-c4d7dd23be96\subagents\workflows\wf_d251ad6c-472\agent-ac7cb3bd47541164e.jsonl`
- SHA-256: `8c4614631df1437e6652f022dd27356651c4192d02d7951582dd36090a4fb095`
- Source modified: `2026-06-08T08:48:06+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_d251ad6c-472`
- session_id: `c6028e72-ebfd-4218-89a8-c4d7dd23be96`

Transcript

1. user (2026-06-08T08:45:41.159Z)

```
You are the deciding architect. Merge these 3 positioning proposals into ONE authoritative
recommendation for how sports-data-collector should be positioned to do manual ingestion of
rally-annotator golden labels. Resolve conflicts explicitly (record them in disagreements with
the call you made). Keep the roadmap prioritized P0->P2 and grounded in the cited files. Prefer
a pragmatic P0 that ships partial-label ingestion soon, while stating the canonical-hub + label-
lifecycle end-state. Do NOT invent files/fields not supported by the findings.

PROPOSALS:
[
  {
    "lens": "PRAGMATIST",
    "positioning_statement": "The lean rally-annotator CSV
(rally_number,start_time,end_time,ending_reason,sport) is ALREADY ingestible today through
`import-local` with zero collector code changes ? its columns match the CSV DictReader in
curate.py:232-298 exactly (decimal seconds, integer rally_number, ending_reason maps 1:1, sport
column is harmlessly ignored). The real \"ship this week\" work is NOT building a new import
path; it is (a) a thin pre-import normalizer/validator that catches the three known landmines
BEFORE they corrupt the DB ? fractional rally_number truncation collision, missing/empty
end_time, and partial coverage ? and (b) recording the labeled-window bounds so the indexer's
eval_partial_labels harness scores fairly against a prefix-labeled video. The rich
rally_review.csv format (mm:ss times, `rally` column, pipe-delimited reasons) is explicitly OUT
of scope for this week; route it through a one-off converter to the lean format rather than
teaching import-local to parse it.",
    "role_in_pipeline": "sports-data-collector is the data PRODUCER. Flow for the partial human
CSV: rally-annotator emits lean CSV -> `python -m src.cli.curate import-local --sport badminton
--video-path <mp4> --annotations-path <rallies.csv>` (curate.py:52-155) parses
(curate.py:232-250), validates temporal consistency (validator.py:29-57), and writes
VideoMetadata(status=CURATED) + badminton_rallies rows into SQLite (db.py:210-237, UPSERT) ->
`curate export` (curate.py:440-536) emits per-video `start,end` CSV + manifest.json -> the
badminton-highlight-indexer eval/training consumes start_time/end_time ONLY (load_golden_gts at
calibrate_wasb.py:34-49; partial-label scoring at eval_partial_labels.py:47-78). Everything
except (start,end) is dropped before the indexer sees it.",
    "schema_mapping": [
      {
        "annotator_field": "rally_number",
        "canonical_field": "BadmintonRally.rally_number (int)",
        "handling": "Direct map via _safe_int(row.get('rally_number'), idx+1) at curate.py:242.
Lean CSV is already 1-based integer, so it round-trips cleanly. RISK: _safe_int does
int(float(s)) (parsing.py:9-19) so a fractional 7.5 silently truncates to 7 and COLLIDES with
rally 7 on PK (video_id, rally_number) -> the DELETE-then-INSERT UPSERT (db.py:210-237) keeps
only the last of the two. Guard: reject/renumber fractional values pre-import (see
partial_label_handling)."
      },
      {
```

```
        "annotator_field": "start_time",
        "canonical_field": "BadmintonRally.start_time (float)",
        "handling": "Direct map via safe_float_required(row.get('start_time')) at curate.py:243.
Decimal seconds expected; lean CSV already decimal. No mm:ss support in this path
(parse_time_string exists in parsing.py:32-49 but import-local does NOT call it), which is fine
because the lean CSV is decimal."
      },
      {
        "annotator_field": "end_time",
        "canonical_field": "BadmintonRally.end_time (float)",
        "handling": "Direct map via safe_float_required(row.get('end_time')) at curate.py:244.
Empty/missing -> ValueError -> whole import aborts (curate.py:84-86). Guard: drop rows with
blank end_time in the normalizer rather than letting one bad row kill the run."
      },
      {
        "annotator_field": "ending_reason",
        "canonical_field": "BadmintonRally.ending_reason (str|None)",
        "handling": "Direct map via row.get('ending_reason') at curate.py:248. Vocabulary
{winner,forced_error,unforced_error,service_fault,let,other} matches the canonical enum exactly.
Stored in DB but DROPPED on export (curate.py:490-494 writes only start,end) ? indexer never
uses it, so no conversion needed."
      },
      {
        "annotator_field": "sport",
        "canonical_field": "(none ? comes from --sport CLI flag)",
        "handling": "IGNORED by import-local (no row.get('sport') in curate.py:240-250). Sport
is taken from the required --sport flag. No validation that CSV sport matches the flag;
acceptable for this week since the operator sets --sport explicitly."
      },
      {
        "annotator_field": "(absent) score_before/score_after/shots_count/last_shot_outcome",
        "canonical_field": "BadmintonRally optional fields",
        "handling": "Lean CSV omits these; _safe_int(..., None)/row.get(...) degrade them to
None (curate.py:245-249). All None, all dropped on export. No action."
      }
    ],
    "partial_label_handling": "THREE concrete issues, each with the smallest fix:\n\n(1)
FRACTIONAL RALLY NUMBERS (e.g. 7.5, 16.5 from corrections): The lean rally-annotator output is
integer-only (rally_annotator.lua hard-codes a monotonic int counter), so files coming straight
from the VLC tool are SAFE today. Fractional numbers only appear in the rich rally_review.csv,
which is out of scope. Guard for safety: add a 3-line check in the pre-import normalizer that
errors loudly if any rally_number string contains '.', because _safe_int silently truncates
(parsing.py:17) and the PK collision (db.py:210-237) destroys data with no error. Do NOT widen
the schema to float this week ? the DB column is INTEGER (db.py:77) and export orders by
rally_number (db.py:543); changing it ripples. If a fractional file MUST be imported, renumber
sequentially (1..N by start_time) in the normalizer before import.\n\n(2) MISSING end_time:
safe_float_required raises (parsing.py:22-28), and one bad row aborts the ENTIRE import
(curate.py:84-86). For partial human labels this is too brittle. Smallest fix: in the
normalizer, drop any row where end_time is blank or end_time <= start_time (this mirrors what
the downstream loader already does ? load_golden_gts skips rows where not e > s,
calibrate_wasb.py:47). This keeps the good rallies and discards the unfinished one rather than
failing the whole video.\n\n(3) PARTIAL COVERAGE (rallies cover only a prefix/window of the
video, e.g. 1-40 of ~100): import-local and the validator (validator.py:29-57) DO NOT care about
contiguity or gaps ? they only check positive duration + no overlap between rally_number-sorted
neighbors. So a partial set imports cleanly AS-IS. The real gap is downstream: a naive full-
video eval would penalize the detector for correct predictions in the UNLABELED tail. The
indexer already solves this with eval_partial_labels.py's --window-start/--window-end
(eval_partial_labels.py:47-78), but those bounds are NOT in the manifest (export drops them).
SMALLEST fix that ships value this week: do NOT modify the export schema. Instead, when running
the partial eval, pass --window-end = max(end_time) of the imported rallies (this is exactly
eval_partial_labels.py:75-76's default), and record that number in a sidecar note. The window is
recoverable from the exported start,end CSV itself (max end), so no manifest change is strictly
required to ship.",
    "provenance_and_licensing": "Self-produced manual labels import as license='Proprietary'
(the import-local default, curate.py and the --license flag). check_license_commercial treats
```

Proprietary as commercial-safe (validator.py:24-27, whitelist includes Proprietary), so status=CURATED + is_commercial=true and the video is included in the default `export` (curate.py:478, commercial_only path). This is correct for owned/self-rated footage. CAUTION grounded in the global working-agreement-attribution note: do not stamp a more permissive license than the footage actually carries ? Proprietary is the honest default for self-owned match video. No annotator/author/timestamp is recorded per rally (db.py:210-237 has no such columns); acceptable this week but means re-importing the same video_id silently overwrites prior labels via DELETE-then-INSERT (db.py:210-237) with no audit trail.",
    "recommended_changes": [
      {
        "title": "Import the lean rally-annotator CSV today via import-local, unmodified",
        "rationale": "The file `Adarsh and Avi.rallies.csv` (columns rally_number,start_time,end_time,ending_reason,sport; integer rallies; decimal seconds; complete pairs) already matches the CSV reader at curate.py:232-250 exactly. Run `python -m src.cli.curate import-local --sport badminton --video-path <mp4> --annotations-path 'Adarsh and Avi.rallies.csv'` then `curate export`. Validator (validator.py:29-57) accepts partial/gapped coverage as-is. Zero code changes. This is the fastest path to a consumable eval label.",
        "effort": "S",
        "priority": "P0",
        "files": [
          "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
          "C:/Users/avidu/Projects/Annotation Setup/Collect/Adarsh and Avi.rallies.csv"
        ]
      },
      {
        "title": "Add a thin pre-import normalizer that drops bad-end rows and rejects fractional rally_number",
        "rationale": "Two real landmines in partial human data: (a) a blank/<=start end_time makes safe_float_required raise and abort the WHOLE import (parsing.py:22-28, curate.py:84-86) ? drop those rows instead (mirroring downstream calibrate_wasb.py:47 which already skips not e>s); (b) a fractional rally_number truncates via int(float()) (parsing.py:17) and collides on PK (video_id,rally_number), so the DELETE-then-INSERT UPSERT (db.py:210-237) silently destroys one rally. Normalizer should error loudly on any rally_number containing '.', or renumber 1..N by start_time. Keep it as a standalone script feeding import-local; do NOT touch the INTEGER schema (db.py:77) or the rally_number-ordered export (db.py:543) this week.",
        "effort": "S",
        "priority": "P0",
        "files": [
          "C:/Users/avidu/Projects/sports-data-collector/src/core/parsing.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py"
        ]
      },
      {
        "title": "Run the partial eval with --window-end = max(end_time) of imported rallies",
        "rationale": "Partial labels cover only a prefix; a full-video eval would penalize correct predictions in the unlabeled tail. The indexer already handles this via eval_partial_labels.py:47-78 (restrict_to_window). The window bound is recoverable from the exported start,end CSV (max end), and eval_partial_labels.py:75-76 already defaults window_end to max GT end. So ship by passing the flag at eval time ? no export/manifest schema change needed this week.",
        "effort": "S",
        "priority": "P1",
        "files": [
          "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/eval_partial_labels.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py"
        ]
      },
      {
        "title": "Write a one-off rally_review.csv -> lean CSV converter (only if the rich file is needed)",
        "rationale": "rally_review.csv uses mm:ss.sss strings, a `rally` column (not rally_number) with fractional ids, and pipe-delimited ending_reason like 'forced_error|other' (rally_review.csv rows 15-16) ? none of which the import-local CSV reader understands (it

expects start_time/end_time decimal and single-value ending_reason, curate.py:243-248). Rather than teaching import-local this format, convert offline: mm:ss->seconds (parse_time_string at parsing.py:32-49 already does this), drop the fractional `rally` ids and renumber 1..N by start_time, take the first token before '|' as ending_reason, drop unverified rows (VERIFIED/ending_consistent=NO) if desired. Output the 5-column lean CSV and feed it to recommendation #1. Only do this if the rich file is actually the source of truth ? the lean file already works.",
        "effort": "M",
        "priority": "P2",
        "files": [
          "C:/Users/avidu/Projects/Annotation Setup/rally_review.csv",
          "C:/Users/avidu/Projects/sports-data-collector/src/core/parsing.py",
          "C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md"
        ]
      }
    ],
    "open_questions": [
      "Is the source of truth for THIS week's import the lean `Adarsh and Avi.rallies.csv` (already importable, recommend shipping that) or the rich `rally_review.csv` (needs the P2 converter)? If lean, skip recommendation #4 entirely.",
      "Does the partial-coverage video have a trajectory.csv (WASB output) co-located? eval_partial_labels.py requires --trajectory separately (it is NOT in the manifest); without it you can import+export labels but cannot run the partial eval scoring.",
      "For re-imports of the same video_id, is silent overwrite (DELETE-then-INSERT, db.py:210-237) acceptable, or is a no-clobber guard wanted? Out of scope for this week but worth a decision before bulk imports."
    ]
  },
  {
    "lens": "CANONICAL-HUB ARCHITECT ? position the sports-data-collector as the single canonical ingestion/validation/registration/export hub for ALL annotation sources (manual VLC self-rating, vendor CVAT, vendor CSV review), normalizing every source into one rally record via clean per-source adapters, with the indexer consuming ONLY the collector's export.",
    "positioning_statement": "The collector should be the ONE place where any rally annotation ? regardless of who produced it or in what shape ? becomes a trusted, registered rally record. Today the architecture is already 80% of the way there: the collector is the sole producer and the indexer consumes only `manifest.json` + per-video `start,end` CSVs (curate.py:440-536; calibrate_wasb.py:34-49 loads start_time/end_time only). But the three sources enter through asymmetric doors. Vendor CVAT flows through a rich adapter (`load_canonical_csv` / `convert` in rally_cvat.py:325-460) that produces all 11 canonical columns and runs structured Issue detection; vendor/manual CSV enters through a SECOND, thinner parser (`_parse_local_annotations` in curate.py:156-302) that reads only 9 fields, tolerates no column aliases, and emits no Issues; and the VLC rally-annotator's lean 5-column CSV (CSV_FORMAT.md, rally_annotator.lua:88) and the 14-column rally_review.csv (rally_review.csv header) have NO dedicated adapter at all ? they are fed raw into the same thin `import-local` path that cannot parse mm:ss strings, fractional rally numbers, pipe-delimited ending_reason, or correction columns. The canonical-hub design makes the collector's `RallyRecord` (rally_cvat.py:81-97) the SINGLE internal normal form, routes EVERY source through a named adapter that emits a `RallyRecord` + `List[Issue]`, runs ALL sources through the SAME validator (validator.py:29-57) and the SAME observability envelope (adapter.py:113-156), and keeps the indexer forever insulated behind the export contract. One source of truth (the SQLite `videos` + `*_rallies` tables, db.py:52-88), one normal form, one validation gate, many thin adapters.",
    "role_in_pipeline": "The collector is the canonical HUB sitting between N annotation producers and exactly one consumer. Producers (left edge): (1) manual VLC rally-annotator ? lean 5-col CSV (rally_annotator.lua:88, CSV_FORMAT.md); (2) the manual review workflow ? rich 14-col rally_review.csv (mm:ss times, fractional rally ids, VERIFIED/ending_consistent flags, true_* correction columns, pipe-delimited ending_reason); (3) vendor Roora ? CVAT XML or canonical CSV (rally_cvat.py:325-460, ROORA_BRIEF.md). Hub (center): each source is normalized by a per-source ADAPTER into the internal `RallyRecord` normal form, validated by the SHARED validator (no-overlap, positive-duration, chronological ? validator.py:29-57), license-gated (check_license_commercial, validator.py:24-27), registered in SQLite with provenance + curation status (db.py:52-88, base.py:24-27), and emitted through the SHARED observability envelope (reporting/adapter.py, spec.yaml). Consumer (right edge): the indexer reads ONLY `curate export`'s `manifest.json` + per-video `start,end` CSVs (curate.py:440-536) and never touches any source format, DB, or rich field ? its eval/training pipelines (calibrate_wasb.py,

distill_local.py:81-87, training.py:50-62, metrics.py match_intervals) consume intervals + manifest metadata (fps, frame_width, local_path) exclusively. This insulation is the load-bearing property: a new sport or a new annotation source is an adapter change at the hub, never a change to the indexer.",
    "schema_mapping": [],
    "partial_label_handling": "Partial labeling is REAL across all three manual sources (rally_review.csv covers only a prefix window; VLC annotator is resumable and a session may stop mid-match ? CSV_FORMAT.md, lua:438-464) but is INVISIBLE to the canonical record and the export. The indexer already has the concept ? `eval_partial_labels.py:47-54` restrict_to_window() drops predictions outside [window_start, window_end] so unlabeled tails are neither rewarded nor penalized ? but window_start/window_end are CLI args (eval_partial_labels.py:68-78), NOT carried in manifest.json. Canonical-hub fix: the hub OWNS the labeled-window as first-class provenance. Each adapter declares the labeled span it covers (default window_start=0.0; window_end = max(end_time) of parsed rallies, matching the indexer default at eval_partial_labels.py:72-82), the collector persists it per video, and `export_eval_set` writes optional `window_start`/`window_end` into each manifest entry alongside fps/num_segments (curate.py:496-509). The indexer then reads the window from the manifest instead of requiring a hand-passed flag ? partial coverage becomes a declared property of the source, set ONCE at the hub. Until that field exists, the hub must NOT silently treat a partial CSV as full coverage: that is exactly how a partially-labeled video poisons precision (every unlabeled-but-real rally the model finds scores as a false positive). Adapters should also surface a `possible_missed_rally`-style Issue (the gap>15s heuristic already exists for CVAT, rally_cvat.py issue detection / INGESTION_PIPELINE.md:68-92) on the manual paths, which today emit no Issues at all (curate.py import-local), so a truncated manual label set is flagged for human confirmation rather than exported blind.",
    "provenance_and_licensing": "Provenance is the hub's job and is currently UNDER-modeled ? this is the biggest gap to a true single-source-of-truth. (1) Annotation SOURCE is invisible: LicenseType.Proprietary (base.py:12-22) is used for both vendor-delivered and manual self-rated imports, and `import-local` hardcodes status=CURATED, bypassing staging (curate.py:117) with no marker distinguishing \"vendor-QA'd at 20% spot-check\" (ROORA_BRIEF.md:79-86) from \"single-rater self-produced, no second POV\". Recommendation: add an `annotation_source` enum to VideoMetadata (e.g. manual_vlc | manual_review | vendor_cvat | vendor_csv) set by the adapter, and stop auto-CURATING manual imports ? route manual/self-rated through STAGING (base.py:24-27 already has the state) so the curation status reflects real QA maturity, not import convenience. (2) No annotator/version/timestamp and no re-tag audit trail: insert_badminton_rallies does DELETE-then-INSERT (db.py:210-237), erasing history on re-import ? the hub should at minimum stamp source + import timestamp per video so a re-annotation is attributable. (3) Licensing is correctly the hub's gate and should STAY there: `check_license_commercial` (validator.py:24-27) + the commercial-only default in export (curate.py:454,478) mean the indexer never sees a non-commercial source unless `--include-noncommercial` is passed ? keep this single committed legal guardrail at the hub and never replicate license logic downstream. Carry license_type/is_commercial into the manifest (already done, curate.py:505-506) so the consumer can audit but never decide.",
    "recommended_changes": [
      {
        "title": "Unify all three ingestion sources behind one adapter interface emitting RallyRecord + List[Issue]",
        "rationale": "Today vendor CVAT runs through the rich adapter (rally_cvat.py convert/load_canonical_csv:325-460, full 11 columns + Issue detection) while manual VLC CSV, rally_review.csv, and vendor CSV all funnel through the SEPARATE thin parser `_parse_local_annotations` (curate.py:156-302) that reads 9 fields, tolerates no aliases, and emits zero Issues. This is two doors into one house. Define a single `RallyAdapter` protocol ? `adapt(path) -> (List[RallyRecord], List[Issue])` ? and implement one adapter per source (VlcLeanAdapter, ReviewSheetAdapter, CvatAdapter, CanonicalCsvAdapter). `import-local` dispatches to the right adapter on the source, then runs the IDENTICAL validate->register->emit-report sequence (curate.py:88-154). RallyRecord (rally_cvat.py:81-97) becomes the one internal normal form for every source; the indexer is untouched.",
        "effort": "L",
        "priority": "P0",
        "files": [
          "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py"
        ]
      },

```
      {
        "title": "Make the manual-CSV adapter tolerate the real rally_review.csv shape (mm:ss
times, fractional ids, pipe-delimited reasons, correction columns)",
        "rationale": "The thin import path (curate.py:232-298) only reads decimal
start_time/end_time, truncates fractional rally_number via safe_int (2.5->2, silent collision
risk), reads no `outcome` alias, and ignores true_start_mmss/true_end_mmss/true_ending overrides
and pipe-delimited 'forced_error|other'. Yet rally_review.csv uses ALL of these
(rally_review.csv header; mm:ss strings like '1:02.062'; fractional rally 7.5/16.5;
VERIFIED/ending_consistent flags). Note `load_canonical_csv` (rally_cvat.py:424-460) and
`_parse_secs` ALREADY handle mm:ss and start_mmss/end_mmss fallback and lowercase headers ? the
review adapter should reuse that helper rather than the weaker curate.py path. Resolve
fractional ids deterministically (e.g. preserve as ordinal, or map to a stable integer with a
recorded original-id note), apply true_* columns as overrides when populated, normalize
VERIFIED/ending_consistent case-insensitively, and split pipe-delimited reasons to primary +
note. This is what lets the rich manual workflow round-trip without manual pre-cleaning.",
        "effort": "M",
        "priority": "P0",
        "files": [
          "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py",
          "C:/Users/avidu/Projects/Annotation Setup/rally_review.csv",
          "C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md"
        ]
      },
      {
        "title": "Run Issue detection on EVERY source, not just CVAT, so manual imports get the
same QA gate",
        "rationale": "`import-local` performs only temporal-integrity validation
(overlap/duration, validator.py:29-57) and emits no structured Issues ? CVAT delivery alone
produces the blocker/high/medium/low Issue stream (rally_cvat.py Issue class;
INGESTION_PIPELINE.md:68-92 with gap>15s missed-rally heuristic). The asymmetry means a self-
produced VLC/review CSV ? which has NO external 20% spot-check (ROORA_BRIEF.md:79-86) and a
single-rater POV ? gets LESS scrutiny than a vendor delivery, exactly inverting where scrutiny
is needed. Lift the Issue detectors (possible_missed_rally, shot_density, vocab, zero_duration,
overlap) to run on RallyRecords from ANY adapter, route the same blockers-block / high-medium-
to-review-sheet logic, and emit through the existing import observability envelope
(reporting/adapter.py:113-156) which today logs only is_valid+errors. One QA gate for all
sources.",
        "effort": "M",
        "priority": "P1",
        "files": [
          "C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/reporting/adapter.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/reporting/spec.yaml"
        ]
      },
      {
        "title": "Record annotation_source provenance and stop auto-CURATING manual self-rated
imports",
        "rationale": "The DB cannot distinguish a vendor-QA'd delivery from a single-rater self-
rating: both land as license=Proprietary (base.py:12-22) and import-local hardcodes
status=CURATED (curate.py:117), bypassing the STAGING state that already exists (base.py:24-27).
For a single source of truth that downstream and humans can trust, the hub must record WHO
produced each annotation set. Add an `annotation_source` field to VideoMetadata (db.py:52-71)
set by the adapter (manual_vlc | manual_review | vendor_cvat | vendor_csv), and route self-
produced manual imports to STAGING by default (promote to CURATED only after the review sheet is
cleared) so curation status reflects real QA maturity. Carry annotation_source into the export
manifest (curate.py:496-509) for downstream auditing ? the indexer reads it but never acts on
it.",
        "effort": "M",
        "priority": "P1",
        "files": [
          "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py",
```

```
          "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py"
        ]
      },
      {
        "title": "Make the labeled window a first-class manifest field so partial labels survive
export",
        "rationale": "Partial/prefix labeling is the common case for manual sources (resumable
VLC sessions; rally_review prefix windows ? CSV_FORMAT.md, lua:438-464), and the indexer already
supports it via restrict_to_window (eval_partial_labels.py:47-54) ? but window_start/window_end
are CLI flags (eval_partial_labels.py:68-78), absent from manifest.json. Without a declared
window the hub silently exports a partial label set as if it were full coverage, so every real-
but-unlabeled rally the model finds is scored as a false positive. The hub should own the
window: each adapter declares it (default 0.0 .. max end_time, matching
eval_partial_labels.py:72-82), the collector persists it per video, and export_eval_set writes
optional window_start/window_end into each manifest entry (curate.py:496-509). The indexer then
reads the window from the manifest instead of a hand-passed flag.",
        "effort": "M",
        "priority": "P1",
        "files": [
          "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py",
          "C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/eval/eval_partial_labels.py"
        ]
      },
      {
        "title": "Keep the indexer behind the intervals-only export contract; document and
freeze it as the single seam",
        "rationale": "The clean property of this architecture is that the indexer consumes ONLY
manifest.json + per-video start,end CSVs (curate.py:440-536; calibrate_wasb.py:34-49 reads
start_time/end_time only; rich fields ending_reason/last_shot_outcome/shots_count/scores are
validated then discarded). This insulation is what makes new sports/sources an adapter-only
change. The risk is contract erosion: header detection is exact-match case-insensitive (a vendor
'start_seconds' would read the wrong column or fail silently ? eval gap). Action: write a single
CONTRACT doc enumerating the export shape (CSV columns start,end decimal seconds; manifest
fields incl. the new window_start/window_end and annotation_source), have BOTH repos reference
it, and add a tiny round-trip test in the collector asserting export matches the indexer loaders
(calibrate_wasb load_golden_gts, batch.py resolve_video_path). Do NOT widen the export with rich
fields the indexer doesn't consume ? keep the seam narrow on purpose.",
        "effort": "S",
        "priority": "P2",
        "files": [
          "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
          "C:/Users/avidu/Projects/sports-data-collector/docs/PIPELINE_ROLE.md",
          "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_wasb.py",
          "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/batch.py"
        ]
      }
    ],
    "open_questions": [
      "Fractional rally ids (7.5, 16.5 in rally_review.csv; 1.1/1.2 lets) collide with the int
PK (video_id, rally_number) in db.py:74-88 and ROORA_BRIEF's no-gaps integer mandate. Should the
canonical schema keep rally_number int and store the original fractional id as provenance, or
widen the key? The hub needs ONE decision so every adapter resolves them identically.",
      "Should rich fields (ending_reason, last_shot_outcome, shots_count, scores) be persisted-
and-exported in a SEPARATE rich manifest for the sibling khelacharya shot-intelligence layer,
while the rally-segmenter export stays intervals-only? The hub already stores them (db.py:74-88)
but export drops them (curate.py:490-494) ? the question is whether a second consumer justifies
a second, wider export rather than widening the one seam the indexer depends on.",
      "Re-tagging today is DELETE-then-INSERT with no history (db.py:210-237). For a true source
of truth, does the hub need annotation versioning / drift detection (old-vs-new ending_reason or
shot_count divergence) when a video is re-annotated, or is last-write-wins acceptable given the
golden-set regression harness (GOLDEN_SET_IMPLEMENTATION.md:51-52) already catches downstream
metric drift?",
      "Should the VLC rally-annotator emit the canonical 11-column CSV directly
```

(rally_annotator.lua:88 currently hardcodes the lean 5-column header), eliminating the need for a lean-vs-rich adapter split, or is the lean format a deliberate low-friction capture format that the hub should keep adapting?"
    ]
  },
  {
    "lens": "LABEL-LIFECYCLE / PROVENANCE PRODUCT ? position the collector as the system of record for WHAT is trustworthy (provenance + license + maturity per source) and HOW MUCH of each video is labeled (the labeled-window), so the indexer's eval/regression restricts to labeled regions and trusts only mature labels automatically.",
    "positioning_statement": "Today the collector is framed as an ingestion-and-export tool: it parses annotations, validates temporal integrity, and emits an intervals-only `start,end` eval CSV plus a manifest (curate.py:440-536). But its real product is the LABEL LIFECYCLE. Two facts about every label are load-bearing downstream yet are NOT captured: (1) how trustworthy a label is ? its provenance (self-rated vs vendor-delivered), its license, and its maturity (needs_review -> confirmed -> gold); and (2) how much of a video is labeled ? the labeled-window, i.e. a video tagged only to t=T with an unlabeled tail. The indexer ALREADY needs both: `eval_partial_labels.py:47-78` invented `restrict_to_window()` precisely because GT covers only a prefix and predictions in the unlabeled tail must be neither rewarded nor penalized ? but `window_start`/`window_end` live only as CLI args, recomputed as `max(end_time)` (eval_partial_labels.py:75-76), not persisted by the collector. And the golden-set regression flow (GOLDEN_SET_IMPLEMENTATION.md GS-2, `backend/eval/regression.py`) grades product-vs-GT, vendor-vs-key, and annotator-vs-annotator off the SAME `List[(start,end)]` the collector exports ? so the collector is the system of record that decides which intervals are gold enough to regress against. Repositioning the collector around the label lifecycle means: it records WHAT is trustworthy (provenance, license, maturity per source) and HOW MUCH of each video is labeled (labeled-window), and the manifest carries both so the indexer restricts eval to labeled windows and trusts only mature labels AUTOMATICALLY, instead of every consumer re-deriving these by hand and risking a false-green regression.",
    "role_in_pipeline": "Producer / system-of-record for ground-truth labels in a two-repo training pipeline (PIPELINE_ROLE.md): the collector curates licensed videos with per-rally [start,end) intervals and exports a manifest.json + per-video CSV (curate.py:440-536) that the indexer consumes for eval, calibration, distillation/training, and golden-set regression. The indexer keys videos by file MD5 and runs against the exact local_path (curate.py:447-448; batch.py:22-40). The collector's distinctive role in the lifecycle: it is the only place that knows a label's provenance (manual import-local self-rating vs Roora CVAT vendor delivery), its license/commerciality gate (validator.check_license_commercial), and ? once extended ? its maturity and labeled extent. It feeds the golden-set FileProvider (GS-1) whose `fetch()` returns the same intervals `metrics.score()` consumes, so whatever the collector marks as gold becomes the regression baseline. Re-tagging a video (re-import) should let the indexer's regress(video, tier, tolerance) flow (GS-2) detect whether the tracked corpus improved or drifted ? but today insert_*_rallies() is DELETE-then-INSERT (db.py:210-237) with no version/audit, so re-tagging silently overwrites history.",
    "schema_mapping": [],
    "partial_label_handling": "CONFIRMED as the single biggest gap and the strongest case for the lens. The labeled-window concept is fully implemented on the indexer side but invisible to the collector: `eval_partial_labels.py:47-54` (restrict_to_window) drops predicted segments wholly outside [window_start, window_end) so the unlabeled tail is neither rewarded nor penalized; `eval_partial_labels.py:75-76` defaults window_end to max(end_time) of GT and window_start to 0.0; both are CLI args, NOT in the manifest or CSV (Collector-Output findings: 'Labeled-window restriction is NOT representable in the current collector export format'). Consequences of the gap: (a) defaulting window_end to max GT end is WRONG when a video is tagged to t=T but the last rally ends well before T (e.g. labeled through a mid-match break) ? real predictions between the last GT rally and T get unfairly penalized as false positives, silently corrupting precision; (b) every consumer must re-supply the window by hand, so the same video can be scored against different windows in different runs, and a golden-set regression baseline (GS-2) captured with one window is not comparable to a re-run with another. The collector is the right owner: it knows from the annotation session (rally-annotator is resumable and 'continues numbering on re-open', README.md:10-14; Lua hard-codes 1-based monotonic numbering) exactly how far a rater got. FIX: add explicit optional `labeled_window_start`/`labeled_window_end` (decimal seconds) per video to VideoMetadata, the DB videos table, and each manifest eval_sets entry; populate from the annotation session extent (or an explicit rater-entered 'labeled up to' marker) rather than inferring from max GT end. The indexer then reads the window from the manifest and applies restrict_to_window automatically ? no CLI guesswork. This is a small, additive, backward-compatible change (consumers ignore unknown manifest keys today) with

```
outsized payoff.",
    "provenance_and_licensing": "Provenance and licensing are PARTIALLY modeled but miss the
dimensions the lifecycle lens needs. What exists: LicenseType enum (base.py:12-22) with a
commercial-safety whitelist (validator.py:24-27) and an is_commercial flag (db.py:65); default
export gates to CURATED + commercial-only (curate.py:450-457, 478-480), with non-commercial kept
in the DB but excluded unless --include-noncommercial. What's MISSING for trustworthiness: (1)
No source/provenance marker ? LicenseType includes 'Proprietary' but cannot distinguish a self-
rated manual import (import_local hardcodes status=CURATED, 'bypass staging by default',
curate.py:109-122) from a Roora vendor delivery that passed the validate-delivery QA gate with a
20% spot-check (INGESTION_PIPELINE.md, ROORA_BRIEF.md:79-86). These two have very different
trust profiles yet land identically as CURATED+Proprietary+commercial-safe. The DB/Provenance
findings flag this directly: 'no source marker (self-rated vs vendor-delivered)'. (2) No
annotator/author/timestamp ? neither VideoMetadata nor the rally schemas record who labeled or
when (DB/Provenance gap), so IAA and re-tag drift can't be attributed. (3) Self-rated
Proprietary is auto-treated as commercial-safe with no rationale/audit (validator.py:21). FIX
direction: add a `provenance`/`source` field (e.g. self_rated | vendor_delivered | imported_3p)
and optional `annotator_id` + import `timestamp` to the videos table and manifest, so the
indexer can weight or filter by source trust and the golden-set flow can require vendor-
delivered+spot-checked sources for the gold tier. The license gate itself is sound; what's
absent is the record of WHO produced the label and HOW it was QA'd ? exactly the 'what is
trustworthy' half of the lens.",
    "recommended_changes": [
      {
        "title": "Add a segment/video maturity ladder (needs_review -> confirmed -> gold)
distinct from the staging workflow",
        "rationale": "CurationStatus (base.py:24-27) is only STAGING|CURATED|REJECTED and is a
video-level WORKFLOW state, not a label TRUST state; the DB/Provenance findings confirm 'no
segment-level maturity ... needs_review -> confirmed -> gold not trackable'. The golden-set
regression flow (GS-2, GOLDEN_SET_IMPLEMENTATION.md) needs to know which labels are gold enough
to be a regression baseline vs which are unreviewed ? otherwise an unconfirmed re-tag can
silently become the answer key and turn a regression false-green (the exact failure GS-1 guards
against by raising on empty fetch, line 49). Add a `maturity` enum (needs_review|confirmed|gold)
on the video (and ideally allow per-segment override) in VideoMetadata + videos table + manifest
eval_sets entry; default export should expose maturity so the indexer can restrict the gold tier
to maturity=gold automatically. import_local should set needs_review (not auto-CURATED) for
self-rated sources, letting promotion to confirmed/gold be an explicit, audited step.",
        "effort": "M",
        "priority": "P0",
        "files": [
          "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
          "C:/Users/avidu/Projects/badminton-highlight-
indexer/docs/GOLDEN_SET_IMPLEMENTATION.md"
        ]
      },
      {
        "title": "Persist the labeled-window (labeled_window_start/end) per video in the DB and
manifest",
        "rationale": "The labeled-window is implemented on the indexer
(eval_partial_labels.py:47-78) but lives only as CLI args, with window_end defaulting to max GT
end (line 75-76) ? wrong when a video is tagged to t=T past the last rally, silently penalizing
valid predictions in the labeled-but-rally-free tail. The collector knows the true labeled
extent from the (resumable, monotonic) annotation session (rally-annotator README.md:10-14). Add
optional labeled_window_start/labeled_window_end (decimal seconds) to VideoMetadata, the videos
table, and each manifest eval_sets entry so the indexer applies restrict_to_window automatically
and reproducibly. Additive and backward-compatible (consumers ignore unknown keys).",
        "effort": "M",
        "priority": "P0",
        "files": [
          "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
          "C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/eval/eval_partial_labels.py"
```

```
        ]
      },
      {
        "title": "Record label provenance (source) + annotator + timestamp on every video",
        "rationale": "LicenseType/'Proprietary' cannot distinguish a self-rated manual import
(import_local hardcodes CURATED, curate.py:109-122) from a QA-gated Roora vendor delivery
(INGESTION_PIPELINE.md validate-delivery, 20% spot-check ROORA_BRIEF.md:79-86); DB/Provenance
findings explicitly note 'no source marker (self-rated vs vendor-delivered)' and 'no
annotator/author metadata ... no timestamp for import or re-tagging'. Without this the 'what is
trustworthy' half of the lens is unrepresentable and IAA/drift cannot be attributed. Add a
`provenance`/`source` enum (self_rated|vendor_delivered|imported_3p), optional annotator_id, and
an import/updated_at timestamp to the videos table + VideoMetadata + manifest, so the indexer
and golden-set flow can weight/filter by source trust.",
        "effort": "M",
        "priority": "P1",
        "files": [
          "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py"
        ]
      },
      {
        "title": "Replace DELETE-then-INSERT re-tag with a versioned, audited re-tag path
feeding the regression flow",
        "rationale": "insert_badminton_rallies() clears then bulk-inserts (db.py:210-237),
erasing the prior labels for a video_id with no version, diff, or audit (DB/Provenance: 'No re-
tagging audit trail ... erases history'; 'No label quality regression tracking'). This is
squarely against the lens's 'so re-tagging improves a tracked corpus' goal: the golden-set
regress(video, tier, tolerance) (GS-2) is built to tell improvement from drift, but it can only
do so if the OLD labels survive as a comparable baseline. Keep a label version/history (or at
least a prior-snapshot) on re-import and surface a re-tag event (counts changed, boundaries
moved) so a re-tag can be regressed against its predecessor rather than silently overwriting it.
Pairs naturally with the maturity ladder (re-tag resets a gold label to needs_review until re-
confirmed).",
        "effort": "L",
        "priority": "P1",
        "files": [
          "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
          "C:/Users/avidu/Projects/badminton-highlight-
indexer/docs/GOLDEN_SET_IMPLEMENTATION.md"
        ]
      },
      {
        "title": "Emit structured QA flags on import-local (not just an error string list),
mirroring the vendor delivery gate",
        "rationale": "The CVAT delivery path emits structured Issue objects (severity
blocker/high/medium/low + code) and a human review sheet (rally_cvat.py;
INGESTION_PIPELINE.md:68-92), but import-local only runs temporal validation and logs errors as
a plain string list (DB/Provenance: 'Import validation does not emit observability flags';
'local vs vendor validation asymmetry'). For the lifecycle to record HOW MUCH to trust a self-
rated label, import-local should surface the same structured flags (gaps, shot_density,
possible_missed_rally, off-vocab ending_reason) into the maturity decision ? e.g. flagged labels
cannot auto-promote past needs_review. This makes maturity earned by evidence rather than
asserted.",
        "effort": "M",
        "priority": "P2",
        "files": [
          "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py",
          "C:/Users/avidu/Projects/sports-data-collector/src/reporting/spec.yaml",
          "C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py"
        ]
      },
      {
```

```
        "title": "Carry maturity + provenance + window through export into the manifest contract
and document it",
        "rationale": "export_eval_set already emits per-video
license_type/is_commercial/status/num_segments into the manifest (curate.py:496-509); the lens's
payoff only lands if maturity, provenance, and labeled-window ride alongside them so the
indexer's eval and golden-set regression can filter ('gold + vendor_delivered only') and window-
restrict automatically without CLI guesswork. Extend the manifest eval_sets entry with maturity,
provenance, annotator_id, labeled_window_start/end, and document the enriched contract so
consumers (calibrate_*, distill_local, regression) can adopt it. Note the by-design narrowing:
the eval CSV stays intervals-only (start,end), and ending_reason/last_shot_outcome/shots_count
remain DB-only and unused downstream (Collector-Output gap) ? so this enrichment belongs in the
MANIFEST, not the CSV.",
        "effort": "S",
        "priority": "P2",
        "files": [
          "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
          "C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/eval/eval_partial_labels.py",
          "C:/Users/avidu/Projects/badminton-highlight-
indexer/docs/GOLDEN_SET_IMPLEMENTATION.md"
        ]
      }
    ],
    "open_questions": [
      "Granularity of maturity: video-level only, or per-segment? The golden-set keystone scores
whole-video interval lists (GOLDEN_SET_IMPLEMENTATION.md:25), so video-level maturity is the
cheapest unlock; per-segment maturity (a single rally confirmed while others need review) is
richer but the DB PK is (video_id, rally_number) so it would need a new column on each child
table.",
      "Who sets the labeled-window: explicit rater action ('I labeled up to mm:ss') vs inferred
from the last annotated rally end? Inferring is automatic but reproduces the
eval_partial_labels.py:75-76 bug for videos labeled past the last rally; an explicit marker is
correct but needs UI/CLI support in rally-annotator and import-local.",
      "Should re-tag history be full versioning or just a single prior snapshot? Full versioning
best serves drift analysis but adds schema/storage; a single prior snapshot is enough to let
GS-2 regress new-vs-previous and is far cheaper.",
      "Does the gold tier require a specific provenance (vendor_delivered + spot-checked) or can
a thoroughly self-reviewed manual label reach gold? This sets whether maturity and provenance
are independent axes or maturity is partly derived from provenance.",
      "Fractional rally_number (7.5, 16.5 from corrections; rally_review.csv) is truncated by
safe_int and unsupported by the int PK (Manual-Artifacts + Local-Ingestion gaps). Re-tagging
that inserts a corrected sub-rally currently can't be represented ? does the maturity/re-tag
work need to resolve fractional numbering first, or is that a separate track?"
    ]
  }
]

SUBSYSTEM FINDINGS:
[
  {
    "subsystem": "Sports Data Collector - Local/Manual Ingestion CLI",
    "summary": "The sports-data-collector provides a manual ingestion pipeline (`import-local`
command) that accepts local video files paired with JSON or CSV annotation files in a sport-
specific rally/point/delivery format. It parses annotations, validates temporal consistency (no
overlaps, ordered progression), registers metadata + segments in SQLite, and exports curated
data as indexer-ready eval CSVs containing only (start, end) decimal-second intervals per
video.",
    "key_facts": [
      "CLI entry: src/cli/curate.py:844-961 (main function with subparser setup); import-local
command registered line 854-863",
      "import_local() method: src/cli/curate.py:52-155; accepts --sport (required, lowercase),
--video-path (required), --annotations-path (required), --match-name (optional, defaults to
video basename), --video-id (optional, defaults to lowercased match_name), --license
(default='Proprietary'), --fps (default=30.0), --resolution (default='1080p')",
      "_parse_local_annotations() method: src/cli/curate.py:156-302; accepts .json or .csv
```

```
files; normalizes CSV headers to lowercase; raises ValueError if unsupported extension",
      "Badminton schema: src/core/schemas/badminton.py:4-22; required fields: video_id,
rally_number (int), start_time (float), end_time (float); optional: score_before, score_after,
shots_count (int), ending_reason, last_shot_outcome; validates end_time > start_time",
      "CSV parsing: src/cli/curate.py:232-298; uses csv.DictReader with normalized lowercase
headers; safe_int() with idx+1 default for rally_number; safe_float_required() for start_time,
end_time (raises ValueError if missing/empty); gracefully degrades optional fields to None",
      "JSON parsing: src/cli/curate.py:161-231; expects list of objects; uses identical
safe_int/safe_float logic; treats missing required fields (start_time, end_time) as ValueError",
      "Validation pipeline: src/cli/curate.py:88-106 calls
validator.validate_badminton_rallies() at src/core/validator.py:29-57; checks: no zero/negative
duration (start_time < end_time), no temporal overlaps between sequential rallies (sorted by
rally_number), chronological progression; returns (bool, List[str]) errors",
      "DB insertion: src/cli/curate.py:123-136 inserts VideoMetadata (status=CURATED for manual
imports, bypassing staging) + rallies; db.insert_badminton_rallies() at src/core/db.py:210-237
performs UPSERT (DELETE then INSERT per video_id)",
      "Export format: export_eval_set() at src/cli/curate.py:440-536; emits per-video CSV with
header [start,end] + decimal-second tuples (queried via get_segment_intervals() at
src/core/db.py:527-546); manifest.json pairs CSVs with video metadata (local_path, video_url,
fps, resolution, etc.)",
      "License handling: LicenseType enum at src/core/schemas/base.py:12-22 (MIT, Apache-2.0,
BSD, CC0, CC-BY, CC-BY-NC, CC-BY-NC-SA, Public-Domain, Proprietary, Unknown);
validator.check_license_commercial() at src/core/validator.py:24-27 checks against whitelist
(config default: MIT, Apache-2.0, BSD, CC0, CC-BY, Public-Domain, Proprietary)",
      "CurationStatus enum: src/core/schemas/base.py:24-27 (STAGING, CURATED, REJECTED); manual
imports set status=CURATED directly (line 117)",
      "Rally-annotator CSV format compatibility: docs/CSV_FORMAT.md specifies columns
(rally_number, start_time, end_time, ending_reason, sport); decimal seconds, 1-based rally
numbers; sports-data-collector import-local reads this directly (compatible via DictReader
header matching)"
    ],
    "contracts": [
      "import-local CLI flags: --sport (required:
badminton|tennis|cricket|pickleball|tabletennis), --video-path (required: file path),
--annotations-path (required: .json or .csv), --match-name (str, optional), --video-id (str,
optional), --license (str, default='Proprietary'), --fps (float, default=30.0), --resolution
(str, default='1080p')",
      "CSV input columns (required): rally_number (int or auto-indexed), start_time (float,
decimal seconds), end_time (float, decimal seconds); optional: score_before, score_after,
shots_count, ending_reason, last_shot_outcome. Case-insensitive header matching via lowercase
normalization.",
      "JSON input structure: list of objects, each with keys: rally_number (int), start_time
(float), end_time (float), plus optional fields matching CSV",
      "Badminton rally object contract: video_id (str), rally_number (int), start_time (float),
end_time (float); start_time must be < end_time; rally_number serves as chronological sort key",
      "DB schema: badminton_rallies table (video_id, rally_number, start_time, end_time,
score_before, score_after, shots_count, ending_reason, last_shot_outcome) with PK=(video_id,
rally_number), FK on videos.video_id with ON DELETE CASCADE",
      "Export format: per-video CSV at <out_dir>/<sport>/<video_id>.csv with header [start,end]
and (start_time, end_time) decimal-second tuples in chronological order; manifest.json with
entries {video_id, sport, csv, local_path, video_url, match_name, fps, resolution, license_type,
is_commercial, status, num_segments}"
    ],
    "gaps_or_mismatches": [
      "Fractional rally_number support: rally-annotator can produce decimal rally_number (e.g.,
2.5 for resegmented rallies); sports-data-collector expects int, converts via safe_int() which
truncates (2.5 -> 2), causing loss of distinction between fractional variants; no explicit error
on truncation",
      "Partial coverage handling: rally-annotator CSV may have gaps (e.g., rally 1,2,4 missing
3) or float numbers (1.1, 1.2); validator checks only temporal non-overlap and chronological
order via rally_number sort, not contiguity or valid numbering schemes",
      "Missing optional fields: sport column in rally-annotator CSV is self-documenting but
ignored by import-local; no validation that CSV sport matches --sport flag",
      "Score/outcome fields: rally-annotator tracks ending_reason enum
(winner|forced_error|unforced_error|service_fault|let|other); maps exactly to
```

```
BadmintonRally.ending_reason (optional); no score_before/score_after in rally-annotator, must be
None in import",
      "Null/empty handling: safe_float_required() treats empty string as ValueError; optional
int fields default to None (not 0) via safe_int(..., None) if missing; no distinction between
truly-missing and user-entered null",
      "Time format flexibility: parser supports only decimal seconds (float); does NOT support
mm:ss or HH:MM:SS time strings (unlike some consuming tools), only raw float values"
    ],
    "files": [
      "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/cli/ingest.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/core/parsing.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/badminton.py",
      "C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md"
    ]
  },
  {
    "subsystem": "Canonical Rally Schema & CSV I/O (sports-data-collector)",
    "summary": "The sports-data-collector defines a canonical rally record schema
(BadmintonRally/PickleballRally/TableTennisRally) with required fields video_id, rally_number,
start_time, end_time, and optional fields score_before/after, shots_count, ending_reason,
last_shot_outcome. Validation enforces non-negative start times, strictly positive durations,
chronological ordering, and no temporal overlaps. The canonical CSV (ROORA_BRIEF format)
includes 11 columns: rally_number, start_mmss, end_mmss, start_time, end_time, shots_count,
ending_reason, last_shot_outcome, score_before, score_after, notes. CVAT XML is converted to
this canonical format; import-local CSV ingest and export to eval sets both read/write canonical
columns only, dropping all other fields.",
    "key_facts": [
      "BadmintonRally schema: C:/Users/avidu/Projects/sports-data-
collector/src/core/schemas/badminton.py:4-22. Required: video_id (str), rally_number (int),
start_time (float), end_time (float). Optional: score_before/score_after (str), shots_count
(int), ending_reason (str), last_shot_outcome (str).",
      "Validation: C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py:29-57
(validate_badminton_rallies). Rules: start_time >= 0, end_time > start_time
(Pydantic+validator), no temporal overlap between consecutive rallies, sorted by rally_number.",
      "Canonical CSV read: C:/Users/avidu/Projects/sports-data-
collector/src/parsers/cvat/rally_cvat.py:424-460 (load_canonical_csv). Reads decimal
start_time/end_time or mm:ss start_mmss/end_mmss; normalizes headers to lowercase; skips rows
missing rally_number or usable times; frame_min/frame_max set to -1 (CSV has no frame info).",
      "Canonical CSV write: C:/Users/avidu/Projects/sports-data-
collector/src/parsers/cvat/rally_cvat.py:356-372 (write_canonical_csv). 11 columns:
rally_number, start_mmss, end_mmss, start_time (%.3f), end_time (%.3f), shots_count,
ending_reason, last_shot_outcome, score_before, score_after, notes. Empty fields written as
blank strings.",
      "Canonical CSV columns (ROORA_BRIEF): C:/Users/avidu/Projects/sports-data-
collector/docs/annotation/ROORA_BRIEF.md:18-30. Enum ending_reason: 'winner' | 'forced_error' |
'unforced_error' | 'service_fault' | 'let' | 'other'. Enum last_shot_outcome: 'in' | 'net' |
'out' | 'n_a' (n_a for service_fault/let).",
      "Import-local CSV parse: C:/Users/avidu/Projects/sports-data-
collector/src/cli/curate.py:232-250. Reads CSV with DictReader, normalizes headers lowercase,
uses safe_int/safe_float for optional fields (defaults to None), required fields
(start_time/end_time) raise ValueError if missing. No sport/status/maturity/fractional columns
read; only 9 fields map to BadmintonRally.",
      "Export eval set: C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py:440-536.
Writes per-video CSV with 2 columns: start, end (decimal seconds only). No rally_number,
ending_reason, last_shot_outcome, shots_count, or scores in export; uses get_segment_intervals
(start_time, end_time only from database).",
      "CVAT conversion: C:/Users/avidu/Projects/sports-data-
collector/src/parsers/cvat/rally_cvat.py:325-353 (convert). Time recomputed from
frame_min/frame_max: start_time = frame_min / fps, end_time = frame_max / fps (vendor
start_time/end_time attributes discarded). Issues reported (not auto-fixed): zero_duration,
ending_reason_vocab, outcome_vocab, shot_density, frame_out_of_range, overlap,
```

possible_missed_rally.",
      "Issue detection allowed vocabularies: C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py:50-61. DEFAULT_ALLOWED_REASONS: {'winner', 'forced_error', 'unforced_error', 'service_fault', 'let', 'other'}. DEFAULT_ALLOWED_OUTCOMES: {'in', 'net', 'out', 'n_a'}.",
      "Database schema: C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:74-88. badminton_rallies table: video_id (TEXT), rally_number (INTEGER), start_time (REAL), end_time (REAL), score_before (TEXT), score_after (TEXT), shots_count (INTEGER), ending_reason (TEXT), last_shot_outcome (TEXT). PK: (video_id, rally_number).",
      "Canonical column set on read (CSV / CVAT): rally_number, start_time (or start_mmss), end_time (or end_mmss), shots_count, ending_reason, last_shot_outcome, score_before, score_after, notes. Headers normalized to lowercase; missing optional fields degrade to None.",
      "Canonical column set on write (CSV export): ONLY start, end (decimal seconds). Rally detail, ending_reason, shots_count, outcomes, scores, and video metadata NOT exported. Separate manifest.json carries video_id, sport, num_segments, local_path, fps, resolution, license, status.",
      "Canonical CSV in curate.py: C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py:559 (write_canonical_csv arg). Called by convert-cvat command; output CSV imports via import-local.",
      "Vendor comparison: C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/eval_partial_labels.py (partial labels harness) and C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_wasb.py:34-49 (load_golden_gts). Both read CSV with start_time, end_time columns ONLY; case-insensitive header matching; no requirement for rally_number, ending_reason, shots_count, scores, or outcome fields."
    ],
    "contracts": [
      "BadmintonRally (& PickleballRally, TableTennisRally): video_id: str (required), rally_number: int (required, ?1, chronological), start_time: float (required, ?0), end_time: float (required, >start_time), score_before: str | None (default None), score_after: str | None (default None), shots_count: int | None (default None), ending_reason: str | None (default None), last_shot_outcome: str | None (default None). Pydantic validation: start_time ? 0; end_time > start_time.",
      "Canonical CSV columns (on read from CVAT/import-local): rally_number (int, required), start_time (float, required | start_mmss fallback), end_time (float, required | end_mmss fallback), shots_count (int | None), ending_reason (str | None, vocab: winner/forced_error/unforced_error/service_fault/let/other), last_shot_outcome (str | None, vocab: in/net/out/n_a), score_before (str | None, format: 'A-B'), score_after (str | None, format: 'A-B'), notes (str | None). Header normalization: all headers lowercased and stripped; field matching is lenient (aliases for last_shot_outcome: outcome, last_shot_outcome (in/net/out)).",
      "Canonical CSV columns (on write to export CSV): ONLY start (decimal seconds, float), end (decimal seconds, float). NO rally_number, ending_reason, shots_count, scores, or outcome. Written as 2-column CSV with header row.",
      "Canonical CSV columns (on write to CVAT-converted CSV): 11 columns in order: rally_number, start_mmss (format m:ss.mmm), end_mmss, start_time (format %.3f), end_time (format %.3f), shots_count (empty string if None), ending_reason (empty if None), last_shot_outcome (empty if None), score_before (empty if None), score_after (empty if None), notes (empty if None).",
      "Validation (validate_badminton_rallies): Returns (bool, List[str] errors). Checks: start_time ? 0, end_time > start_time, sorted chronologically by rally_number, no temporal overlap (end[i] <= start[i+1] for consecutive rallies). Blocker errors: timestamp violations, overlaps. Both CVAT and CSV paths feed this validator.",
      "CVAT issue detection (detect_issues): Returns List[Issue] with severity ('blocker' | 'high' | 'medium' | 'low'), code (zero_duration, ending_reason_vocab, outcome_vocab, shot_density, frame_out_of_range, overlap, possible_missed_rally), message (human-readable), rally_number (optional). Blockers: zero_duration, overlap. NOT: sampling density (step/fps), since downstream padding absorbs ~0.5s uncertainty.",
      "Ending reason vocabulary (canonical enum): {'winner', 'forced_error', 'unforced_error', 'service_fault', 'let', 'other'}. Shared across badminton, tennis, table tennis, pickleball, padel. Last-shot-outcome is SEPARATE and orthogonal (winner can have outcome='in'; error can have outcome='net'/'out'; service_fault/let have outcome='n_a').",
      "Import-local CSV format: flexible headers (case-insensitive), supports start_mmss/end_mmss OR start_time/end_time (mm:ss parsed via formula or direct float). rally_number defaults to 1-based row index if missing. shots_count, ending_reason,

```
last_shot_outcome, score_before, score_after, notes are all optional (degrade to None). No
sport/status/maturity/curation-level fields read; those come from CLI args."
    ],
    "gaps_or_mismatches": [
      "ending_reason is PRESENT in canonical schema but NOT in eval CSV export (badminton-
highlight-indexer loads only start_time/end_time from golden CSVs). Implication: the rally-
indexer evaluation harness does NOT use ending_reason for scoring; it treats rallies as
unlabeled intervals.",
      "last_shot_outcome is PRESENT in canonical schema but NOT in eval CSV export (same as
ending_reason). The indexer calibration does not distinguish rally outcomes.",
      "shots_count is OPTIONAL in canonical schema and in CSV import but NOT captured in export
CSVs. Database stores it; export drops it. Unclear if downstream uses this field.",
      "Fractional rally numbers (e.g., 1.1, 1.2 for replays/lets) are NOT supported by the
canonical schema (rally_number is int). ROORA_BRIEF mandates integer 1-based rally_number with
no gaps. If vendors annotate sub-rallies (e.g., 1st serve fault = 1, 1st let = 1.1, 2nd serve =
1.2), the integer constraint will reject them or require manual splitting.",
      "Curation status / maturity metadata (e.g., STAGING vs CURATED, confidence flags, rework
needed) are NOT in the canonical CSV. They are stored in the database (videos.status enum:
staging/curated/rejected) but do NOT round-trip through CSV export. Annotator/vendor CSVs have
no analogous columns.",
      "Missing end_time is NOT explicitly handled by the canonical schema. ROORA_BRIEF mandates
end_time > start_time always. If a vendor omits end_time or provides end_time <= start_time,
load_canonical_csv skips the row silently (no error logged). Import-local raises ValueError if
end_time is missing/unparseable.",
      "Annotation source / vendor / annotator ID is NOT in the canonical CSV. Video metadata
carries match_name and video_id but no annotator/rework-status/version tracking per rally.",
      "The canonical CSV is NARROWER on export than on import: export loses ending_reason,
last_shot_outcome, shots_count, and scores. A CSV exported from the database and re-imported
would preserve only start_time/end_time; all enrichment is lost. This is by design (eval set is
intervals-only) but breaks round-trip fidelity.",
      "CVAT attribute aliases are LENIENT (ending_reason, outcome, last_shot_outcome
(in/net/out) all map to canonical last_shot_outcome), but CSV import/export do NOT tolerate
variant column names. If a vendor-submitted CSV has 'outcome' instead of 'last_shot_outcome', it
is silently ignored.",
      "Frame-based metadata (frame_min, frame_max, n_boxes in CVAT RallyRecord) is NOT preserved
in canonical CSV. Once converted to decimal seconds, frame info is lost.
RallyRecord.load_canonical_csv sets frame_min/max to -1 as a marker."
    ],
    "files": [
      "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/badminton.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/pickleball.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/tabletennis.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/core/parsing.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/parsers/badminton/coachai.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
      "C:/Users/avidu/Projects/sports-data-collector/docs/annotation/ROORA_BRIEF.md",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_wasb.py",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/eval_partial_labels.py",
      "C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md",
      "C:/Users/avidu/Projects/rally-annotator/docs/ENDING_REASONS.md"
    ]
  },
  {
    "subsystem": "Sports Data Collector ? CVAT Vendor Ingestion & Pipeline Role",
    "summary": "The collector is the data producer in a two-repo training pipeline. It curates
sports videos with per-rally temporal annotations [start, end) and commercial-license metadata,
then exports to the badminton-highlight-indexer for processing and offline rally-segmenter
training. The Roora vendor ingestion flow converts CVAT XML exports (or canonical CSVs) through
a four-command pipeline: convert-cvat (normalizing frame?seconds), validate-delivery (QA gate
with blocker checks + human review sheet), import-local (SQLite registration), and export
(indexer-ready CSV+manifest). Manual self-rating via the VLC rally-annotator tool provides an
```

alternative, lightweight ingestion path with minimal metadata. Golden-set role: temporal-only annotations for segmentation evaluation and training, explicitly not for frame-level shuttle detection; non-commercial sources excluded from default exports.",
    "key_facts": [
      "Collector role (PIPELINE_ROLE.md:1-46): data producer feeding badminton-highlight-indexer; goal is offline rally segmenter trained on commercially-licensed, annotated corpus to replace Gemini API",
      "Roora vendor pipeline (INGESTION_PIPELINE.md:7-44): four sequential commands: convert-cvat ? validate-delivery ? import-local ? export; converts CVAT XML boxes+attributes to canonical CSV with frame?seconds recomputation",
      "Critical frame-to-time conversion (rally_cvat.py:24-29): recomputes from frame_min/frame_max using true fps; vendor's start_time/end_time attributes explicitly ignored to avoid systematic errors from step-sampled jobs with wrong fps assumptions",
      "Canonical CSV schema (ROORA_BRIEF.md:14-30): vendor provides rally_number, start_mmss, end_mmss, shots_count, ending_reason, last_shot_outcome, score_before/after, notes; collector auto-fills start_time/end_time as decimal seconds",
      "Validation rules (INGESTION_PIPELINE.md:68-92): blocker issues (overlaps, zero-duration) gate the import; high/medium issues (shot_density, possible_missed_rally, vocab) populate human review sheet; missed-rally detection via gap heuristic (>15s) and inflated-span flagging",
      "Intake manifest fields (INTAKE_MANIFEST_TEMPLATE.csv:1-5): video_id, sport, format, fps (critical), duration_mmss, expected_rallies, assigned_to, status; fps provided to vendor for frame?seconds formula",
      "VLC rally-annotator CSV (rally-annotator/README.md:10-14): minimal 5-column format (rally_number, start_time, end_time, ending_reason, sport) with decimal seconds only; no shots_count/scores; resumable (continues numbering on re-open)",
      "Rally-annotator ingestion (CSV_FORMAT.md:26-41): direct import via curate import-local; optional fields populated as None; validation still runs (overlaps, positive duration); no external QA gate (self-produced)",
      "Quality bar (ROORA_BRIEF.md:79?86): ±0.3s boundary tolerance, zero missed rallies, schema compliance, shots_count exact for ?15 shots or ±1 for longer rallies, 20% human spot-check on every delivery",
      "Golden-set temporal-only contract (ADR-002, GOLDEN_SET_IMPLEMENTATION.md:6-8): (video_id, ordinal, start_time, end_time) shape shared across five sports; NOT used for shuttle detector training (lacks per-frame x,y labels); only for temporal segmenter training + eval",
      "Regression testing (GS-2, GOLDEN_SET_IMPLEMENTATION.md:51-52): regress(video, tier, tolerance) compares predictions vs GT, flags when precision/recall drop >5% default; baseline snapshots per video with re-snapshot path for legitimate improvements",
      "Non-commercial exclusion (INGESTION_PIPELINE.md:124-129): videos kept in DB regardless of license; default export commercial-only; non-commercial sources excluded unless --include-noncommercial flag passed",
      "Observability reports (INGESTION_PIPELINE.md:131-144): per-video JSON (machine-readable), technical .md, and customer summary .md emitted on every validate-delivery/convert-cvat/import-local run for debugging/transparency"
    ],
    "contracts": [
      "Video identity contract: indexer keys by file MD5; collector by human video_id; manifest carries exact local_path so downstream processing runs on that exact file (acquisition precedes processing)",
      "CSV schema contract: collector exports per-video (start, end) CSVs matching indexer's eval format; manifest.json pairs each CSV with video file; sport-agnostic (start_time, end_time) shape across badminton, tennis, cricket, pickleball, tabletennis",
      "Canonical CSV format: rally_number (1-based, no gaps), start_time/end_time (decimal seconds), shots_count (optional int), ending_reason (enum: winner/forced_error/unforced_error/service_fault/let/other), last_shot_outcome (enum: in/net/out/n_a), score_before/after (optional 'A-B'), notes (required when ending_reason=other)",
      "CVAT attribute aliases: lenient matching for rally metadata (e.g. 'score_before (a-b)' ? 'score_before'); frame-derived times override vendor-supplied times; frame range [frame_min, frame_max] at true fps = [start_time, end_time] seconds",
      "Validation blockers: overlaps (end[N] > start[N+1]), zero-duration (end ? start), frame_out_of_range; violations exit non-zero and block import; non-blocker issues populate review sheet for human verification",
      "Rally-annotator CSV format: rally_number, start_time (decimal), end_time (decimal), ending_reason, sport; optional columns may be added; consumers read by header name and skip rows where end ? start",

```
      "Commercial license gate: is_commercial flag per video; default export includes only
commercial sources; non-commercial explicitly excluded unless --include-noncommercial passed",
      "Temporal-only annotation contract: (video_id, ordinal, start_time, end_time) shared
across sports; no per-frame (x, y) shuttle coordinates; trained for segmentation (active vs.
dead play), not detection; licensed/owned sources only"
    ],
    "gaps_or_mismatches": [
      "Partial-label strategy not explicitly documented: collector emphasizes 'zero missed
rallies' as non-negotiable but offers no formal framework for handling genuinely ambiguous clips
(warm-up detection, occluded play, administrative stoppages marked 'other')",
      "Rally-annotator metadata subset: VLC tool omits shots_count, scores, notes ? fields that
vendor includes; no documented pathway to enrich rally-annotator CSVs with post-hoc shot counts
or scores before/after import",
      "Missed-rally detection limitations acknowledged but not solved: gap heuristic (>15s) and
shot_density flags are heuristic only; motion-based second-opinion stated as 'future
enhancement' with no timeline",
      "Quality bar vs. rally-annotator: vendor delivery expects 20% human spot-check; self-
produced rally-annotator has no equivalent QA gate (rater is both producer and verifier, single
POV)",
      "Warm-up/knock-up exclusion rule: golden rule in ANNOTATION_GUIDE.md but no automated
detection; rater must visually distinguish real play from pre-match cooperative rallying (relies
on human judgment)",
      "Frame sampling tolerance: documentation acknowledges ~0.5s sampling slack absorbed by
clip padding but doesn't quantify impact on boundary accuracy for edge-case rallies",
      "ADR-002 (golden set temporal-only) doesn't address: whether multi-annotator IAA (inter-
annotator agreement) will be tracked; single-rater bias not factored into regression tolerance
(ADR-004 reuses existing harness, may need sport-specific thresholds)"
    ],
    "files": [
      "C:/Users/avidu/Projects/sports-data-collector/docs/PIPELINE_ROLE.md",
      "C:/Users/avidu/Projects/sports-data-collector/docs/INGESTION_PIPELINE.md",
      "C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py",
      "C:/Users/avidu/Projects/sports-data-collector/docs/annotation/ROORA_BRIEF.md",
      "C:/Users/avidu/Projects/sports-data-collector/docs/annotation/ANNOTATION_GUIDE.md",
      "C:/Users/avidu/Projects/sports-data-
collector/docs/annotation/INTAKE_MANIFEST_TEMPLATE.csv",
      "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
      "C:/Users/avidu/Projects/rally-annotator/README.md",
      "C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md",
      "C:/Users/avidu/Projects/rally-annotator/docs/ENDING_REASONS.md",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/docs/GOLDEN_SET_IMPLEMENTATION.md",
      "C:/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/decisions/DECISIONS.md"
    ]
  },
  {
    "subsystem": "Manual Rally Annotation Artifacts",
    "summary": "Two distinct CSV formats exist for manual rally annotation: (1) the lean rally-
annotator output with 5 core columns (rally_number, start_time, end_time, ending_reason, sport),
and (2) the richer rally_review format with 14 columns adding metadata for manual curation (shot
counts, time-in-minutes-seconds format, verification flags, reviewer comments). The
ending_reason taxonomy is shared across both: winner, forced_error, unforced_error,
service_fault, let, other. Manual ingesters must handle partial labeling (videos only partially
tagged), fractional rally numbers (7.5, 16.5 from corrections), missing end_time values, status
flags (VERIFIED/needs review), and free-text notes capturing net touchdowns and flagged
anomalies like OVERLAP or INFLATED timing.",
    "key_facts": [
      "rally-annotator CSV columns (5): rally_number (int, 1-based monotonic), start_time
(float, decimal seconds), end_time (float, decimal seconds, always > start_time), ending_reason
(enum: winner|forced_error|unforced_error|service_fault|let|other), sport (enum:
badminton|tennis|table_tennis|pickleball|padel) - CSV_FORMAT.md:6-18",
      "rally_review CSV columns (14): rally (int, rally identifier, may be fractional like 7.5
or 16.5), start_mmss (string, mm:ss.sss format, e.g. '1:02.062'), end_mmss (string, mm:ss.sss
format), dur_s (float, duration in seconds), shots (int, shot count for the rally),
ending_reason (enum or pipe-delimited alternates like 'forced_error|other'), ending_consistent
(yes|NO, verification flag), notes (free-text, e.g. 'net', OVERLAP, INFLATED), auto_flag (status
```

```
field for automated review), VERIFIED(Y/N) (Y/N or empty), true_start_mmss (optional corrected
start), true_end_mmss (optional corrected end), true_ending (optional corrected reason),
reviewer_comment (optional notes) - rally_review.csv header",
      "Ending reason taxonomy (6 values, shared across sports): winner (opponent couldn't return
in-court shot), forced_error (loser missed under pressure), unforced_error (loser missed routine
shot with time+position), service_fault (serve-rule violation), let (rally replayed under
rules), other (occluded/administrative/unclear) - ENDING_REASONS.md:7-9",
      "rally_annotator.lua line 88 hard-codes header:
'rally_number,start_time,end_time,ending_reason,sport\\n'",
      "rally_annotator.lua line 262 writes CSV with %.3f precision for floats:
'd,%.3f,%.3f,%s,%s' (e.g. '1,37.434,72.194,winner,badminton')",
      "Actual lean CSV (Adarsh and Avi.rallies.csv): 40 rallies, rally_number 1-40 sequential,
start_time range 37.434-599.712s, end_time range 72.194-604.962s, all have
winning/error/service_fault labels, all sport=badminton",
      "Actual rich CSV (rally_review.csv): 26 rows, rally column shows integers AND fractional
numbers (1,2,3... up to 26 but stored as rally NOT rally_number), times in mm:ss.sss format
(e.g. '0:03.003', '2:29.649'), shot counts range 1-15, ending_reason mostly single values but
row 16 shows 'forced_error|other' (pipe-delimited alternatives), VERIFIED column has Y, NO, or
empty, notes field captures qualitative flags ('net', 'OVERLAP', 'INFLATED')",
      "Manual ingester realities: (1) PARTIAL LABELING - video may be only partially tagged,
labels cover only a prefix/window of total runtime, (2) FRACTIONAL RALLY NUMBERS -
corrections/insertions create numbers like 7.5 or 16.5, not just integers, (3) MISSING end_time
- some fields may be blank (not shown in sample but possible), (4) STATUS FLAGS - VERIFIED(Y/N),
ending_consistent(yes/NO), auto_flag field mark review state, (5) last_shot_outcome implicit in
ending_reason enum, (6) shots_count as distinct column in review format, (7) FREE-TEXT NOTES -
unstructured annotations like 'net touchdown' or anomaly flags ('INFLATED timing window',
'OVERLAP with prior rally')",
      "rally_review CSV uses time-in-minutes-seconds string format (mm:ss.sss) while rally-
annotator uses decimal-seconds floats (crucial for parsing)",
      "rally-annotator v1.3 supports 5 sports: badminton, tennis, table_tennis, pickleball,
padel (line 85-87 of Lua)",
      "CSV parsing logic in rally_annotator.lua lines 222-250: forgiving parser that preserves
extra columns, treats any leading-integer line as a rally row, skips header and blank lines"
    ],
    "contracts": [
      "LEAN ANNOTATOR CSV (5 columns): rally_number (int, 1-based, monotonic, continues across
reopens), start_time (float, decimal seconds), end_time (float, decimal seconds, > start_time),
ending_reason (enum: winner|forced_error|unforced_error|service_fault|let|other), sport (enum:
badminton|tennis|table_tennis|pickleball|padel)",
      "RICH REVIEW CSV (14 columns): rally (int or float, rally identifier, may be fractional),
start_mmss (string, mm:ss.sss format), end_mmss (string, mm:ss.sss format), dur_s (float,
seconds), shots (int, shot count), ending_reason (string, may be pipe-delimited alternatives
like 'forced_error|other'), ending_consistent (enum: yes|NO), notes (string, free-text),
auto_flag (string, status field), VERIFIED(Y/N) (enum: Y|N|empty), true_start_mmss (string or
empty), true_end_mmss (string or empty), true_ending (string or empty), reviewer_comment (string
or empty)",
      "ENDING_REASON TAXONOMY (6 shared values across all sports): winner, forced_error,
unforced_error, service_fault, let, other",
      "SPORT TAXONOMY (5 supported): badminton, tennis, table_tennis, pickleball, padel",
      "Decimal seconds format in lean CSV: floats with arbitrary precision but typically 3
decimal places (millisecond-level). Example: 37.434, 72.194",
      "Time format in review CSV: string representation mm:ss.sss
(minutes:seconds.milliseconds). Example: '0:03.003', '4:22.762'. Parser must convert to seconds
for comparison with annotator output",
      "Rally number uniqueness: within one video's CSV, rally_number is unique and monotonic.
Fractional numbers (7.5, 16.5) may appear in review CSV when corrections create inserted
records"
    ],
    "gaps_or_mismatches": [
      "TIME FORMAT MISMATCH: rally-annotator outputs decimal-seconds floats (37.434);
rally_review uses mm:ss.sss string format (0:03.003). Ingester must convert review times to
decimal-seconds or vice versa for reconciliation",
      "COLUMN NAME MISMATCH: lean CSV has 'rally_number' (int counter), but rich CSV uses
'rally' (may be fractional). Manual ingester must handle both naming conventions and account for
fractional IDs from edits/corrections",
```

```
      "MISSING end_time: while both sample CSVs show complete (start, end) pairs, the schema
allows blank end_time. Ingester must validate end > start and reject incomplete rallies",
      "FRACTIONAL RALLY NUMBERS: rally-annotator logic assumes 1-based integer sequencing (line
283-286 of Lua), but real-world manual edits/insertions in review CSV introduce 7.5, 16.5-style
identifiers. Ingester must parse floats, not just integers",
      "ENDING_REASON ALTERNATIVES: lean CSV always single enum value; rich CSV row 16 shows
'forced_error|other' (pipe-delimited), indicating human uncertainty or net-touched ambiguity.
Ingester must handle both scalar and pipe-delimited alternatives",
      "OPTIONAL CORRECTION COLUMNS: rich CSV has true_start_mmss, true_end_mmss, true_ending
columns (all empty in sample but semantically present). Ingester must treat these as optional
overrides for the original times/reason if populated",
      "PARTIAL LABELING: neither sample shows a video labeled only to 50% completion, but
ENDING_REASONS.md and Lua code (lines 438-464) reference the pattern. Ingester must accept
incomplete, prefix-window labels (e.g. rallies 1-15 out of 50 total)",
      "STATUS FLAG SEMANTICS: VERIFIED and ending_consistent use inconsistent enum values (Y/N,
yes/NO capitalization varies). Ingester must normalize or handle case-insensitively",
      "FREE-TEXT NOTES CONTENT: notes field captures sport-specific edge cases ('net touchdown'
in badminton, 'wall bounce' for padel) without schema. Ingester must preserve as-is or define a
taxonomy if automated handling is needed",
      "AUTO_FLAG AND REVIEW STATE: auto_flag field (empty in sample) indicates automated quality
checks triggered (e.g. OVERLAP, INFLATED). Ingester must define what flags are actionable vs.
informational"
    ],
    "files": [
      "C:/Users/avidu/Projects/rally-annotator/docs/CSV_FORMAT.md",
      "C:/Users/avidu/Projects/rally-annotator/docs/ENDING_REASONS.md",
      "C:/Users/avidu/Projects/rally-annotator/vlc/rally_annotator.lua",
      "C:/Users/avidu/Projects/Annotation Setup/Collect/Adarsh and Avi.rallies.csv",
      "C:/Users/avidu/Projects/Annotation Setup/rally_review.csv"
    ]
  },
  {
    "subsystem": "Collector Output Contract for Rally Indexer Eval/Training",
    "summary": "The indexer consumes rally ground truth from the collector via a **manifest.json
+ per-video CSV** contract. The manifest (created by `sports-data-collector`'s `curate export`
command) pairs each video with a CSV of decimal-second rally intervals, plus metadata (fps,
resolution, video path, sport, license, curation status). The CSV columns are minimal:
`start,end` in seconds. The indexer's eval and training pipelines use **temporal IoU matching**
(greedy one-to-one at a threshold, default 0.5) to compare predicted rally windows against
ground truth; partial labels are supported only in the dedicated `eval_partial_labels.py`
harness via explicit `--window-start`/`--window-end` flags, not represented in the export format
itself.",
    "key_facts": [
      "calibrate_wasb.py:34-49 / load_golden_gts() ? reads CSV with columns (case-insensitive):
start_time, end_time (decimal seconds); returns sorted list of (start, end) tuples",
      "calibrate_local.py:66-74 / build_videos() ? expects manifest JSON with structure [{name,
trajectory, fps, frame_width, labels}, ...]; labels = path to golden CSV",
      "eval_partial_labels.py:47-54 / restrict_to_window() ? implements labeled-window concept:
filter predictions/GT to [window_start, window_end] interval; predicted segments outside window
are dropped (neither rewarded nor penalized)",
      "eval_partial_labels.py:68-78 ? window_end defaults to max(end_time) of all GT rallies;
window_start defaults to 0.0; both are CLI args not part of CSV/manifest",
      "rally_slicer.py:53-73 / load_rally_labels() ? loads CSV with columns (case-insensitive):
rally_number, start_time, end_time (decimal seconds OR mm:ss/HH:MM:SS); returns dict
rally_number ? (start, end) seconds",
      "distill_local.py:81-87 / load_video() ? reads manifest entry {fps, frame_width,
trajectory, labels}; calls load_golden_gts(labels) to get GT intervals; label_candidates()
matches candidate windows to GT via IoU",
      "training.py:50-62 / label_candidates() ? uses match_intervals() with iou_threshold
(default 0.5); candidate is positive iff it matches exactly one GT interval at IoU >=
threshold",
      "metrics.py:48-79 / match_intervals() ? greedy one-to-one matching: highest-IoU pairs
claimed first, each pred/GT used at most once; only pairs with IoU >= iou_threshold are
considered for matching",
      "metrics.py:15-29 / interval_iou() ? temporal intersection-over-union: overlap/(union)
```

```
both measured as durations in seconds; returns 0.0 for non-overlapping or degenerate intervals",
      "sports-data-collector curate.py:496-509 / export_eval_set() ? manifest entry structure:
{video_id, sport, csv (relative path), local_path, video_url, match_name, fps, resolution,
license_type, is_commercial, status, num_segments}",
      "sports-data-collector curate.py:490-494 ? exports CSV with header 'start,end' and rows
[start_seconds, end_seconds] (decimal, computed from DB intervals); one rally per row",
      "annotations.py:44-76 / load_annotations() ? legacy simple format: reads CSV with optional
'start'/'start_time' header row; columns 1-2 are start/end timestamps (decimal OR colon-
separated MM:SS/HH:MM:SS); blank lines and # comments ignored",
      "calibrate_local.py:19-20 ? manifest expected from collector at: `../sports-data-
collector/data/eval_export/manifest.json` (relative path hint in docs)",
      "batch.py:22-40 / resolve_video_path() ? normalizes manifest local_path: strips './',
converts backslashes to forward slashes, joins with videos_root if relative; cross-platform
handling for Windows drive letters (C:/...)",
      "eval_partial_labels.py:72-82 ? labeled-window enforcement: load_golden_gts() gets all
rallies; manually compute/override window_end from CLI; restrict_to_window() filters both
predictions and GT to in-window region before scoring",
      "distill_local.py:48-69 / build_candidates() ? proposal stage (velocity_thresh=0.005,
merge_gap=1.0, min_dur=0.5) generates many candidates; WASB trajectory parses trajectory.csv
with columns Frame, Visibility, X, Y"
    ],
    "contracts": [
      "CSV (ground truth): columns start,end | format: decimal seconds (float) | one rally per
row | optional header 'start' or 'start_time' | blank/# lines ignored | no other columns
required/consumed",
      "CSV (human annotations, collector intake): columns rally_number, start_time, end_time,
ending_reason, sport [optional: start_mmss, end_mmss, shots_count, score_before, score_after,
last_shot_outcome, notes] | start/end can be decimal seconds OR MM:SS / HH:MM:SS | rally_number
= string (supports inserted IDs like '7.5') | indexer ignores all columns except
start_time/end_time",
      "manifest.json structure: {commercial_only, statuses, total_videos, total_segments,
eval_sets: [{video_id, sport, csv (relative path to start,end CSV), local_path, video_url,
match_name, fps (float), resolution (string or null), license_type, is_commercial (bool),
status, num_segments}, ...]}",
      "Trajectory CSV (WASB input): columns Frame, Visibility, X, Y | parsed by
TrackNetRunner.parse_trajectory_csv() | motion data required only for calibration/distillation,
not for evaluating labels alone",
      "IoU matching contract: temporal intervals (start, end) in seconds | IoU >= threshold ?
match candidate | greedy one-to-one ? TP counted | threshold defaults to 0.5, configurable via
--iou flag",
      "Labeled-window concept (partial labels): window_start, window_end (seconds) | predicted
intervals outside [window_start, window_end) are dropped before scoring | GT intervals assumed
already within window | NOT part of manifest/CSV; applied only in eval_partial_labels.py via CLI
args",
      "Metadata contract (in manifest): fps (float, required for trajectory windowing and
feature extraction) | frame_width (float, required for WASB velocity normalization) | local_path
(required, string, file system path; used as key with file MD5 for cross-repo identification) |
resolution (optional, string like '1080p')",
      "Feature extraction for training: candidate window bounds ? {duration, peak_velocity,
mean_velocity, active_frame_count (YOLO) OR active_ratio (WASB), track_id_count (YOLO) OR
net_crossings (WASB)} | features must be fps/resolution-INVARIANT except where explicitly
normalized by frame_width"
    ],
    "gaps_or_mismatches": [
      "Labeled-window restriction (eval_partial_labels.py concept) is NOT representable in the
current collector export format ? it is enforced only as CLI args to eval_partial_labels.py, not
persisted in manifest/CSV. If collector needs to support partial labels at scale, a new optional
manifest field (e.g., window_start, window_end) would be required.",
      "Rally metadata (ending_reason, last_shot_outcome, score_before/after, shots_count) are
collected and validated by the collector but NOT consumed by indexer eval/training pipelines ?
they are discarded after validation. Only (start_time, end_time) flow downstream.",
      "Trajectory CSV path is NOT in the manifest ? it is supplied separately as a CLI arg
(--trajectory) to calibrate_wasb, calibrate_local, eval_partial_labels, and distill_local. The
distill pipeline requires it to build features but there is no contract guaranteeing it exists
or is co-located.",
```

```
      "Resolution field in manifest is optional (nullable) and rarely used downstream ? fps is
required but resolution is mainly for observability. Feature extraction normalizes by
frame_width instead of resolution string.",
      "Partial-label support assumes GT labels cover a PREFIX of the video only (every rally up
to window_end is labeled, everything after is unlabeled). This assumption is NOT validated or
documented in the manifest ? it is left to the user to enforce via --window-end.",
      "CSV header detection (start, start_time) is case-insensitive but relies on exact column
name matching (no fuzzy matching) ? a vendor sending 'start_seconds' or 'start_frame' would fail
silently or cause the harness to read the wrong column.",
      "No contract for multi-sport data in a single manifest ? eval_sets can contain mixed
sports but most eval/training scripts assume homogeneous sport (e.g., training.label_candidates
does not filter by sport, so mixing sports in one training set would train a sport-agnostic
model).",
      "IoU threshold is a global setting (--iou flag) applied uniformly across all videos in a
manifest ? no per-video or per-sport threshold override supported in the current contract."
    ],
    "files": [
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\eval\\calibrate_wasb.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\eval\\calibrate_local.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\eval\\eval_partial_labels.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\tools\\rally_slicer.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\eval\\distill_local.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\training.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\metrics.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\annotations.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\batch.py",
      "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\run_phase3_eval.py",
      "C:\\Users\\avidu\\Projects\\sports-data-collector\\src\\cli\\curate.py"
    ]
  },
  {
    "subsystem": "Sports Data Collector - DB/Provenance & Observability Layer",
    "summary": "The collector stores video metadata and sport-specific annotations (rallies,
points, deliveries) in SQLite with two-level schema: a central `videos` table holding provenance
(source URL, license, sport, video_id) and curation status (STAGING/CURATED/REJECTED), plus
sport-specific child tables (badminton_rallies, tennis_points, cricket_deliveries, etc.)
containing segment metadata. Observability is layered via obsreport (soft dependency) which
generates per-video reports at import/validation stages, capturing parse/validation/registration
metrics and flags for delivery issues; local imports bypass vendor-issued flags but still emit
reports showing license, commerciality, and segment counts at registration.",
    "key_facts": [
      "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:52-71: videos table schema ?
video_id (PK), sport, license_type (enum: MIT/Apache/BSD/CC0/CC-BY/CC-BY-NC/CC-BY-NC-SA/Public-
Domain/Proprietary/Unknown), license_url, is_commercial (boolean flag), status (enum:
staging/curated/rejected), match_name, fps, resolution; foreign keys ensure on-delete cascade to
child tables.",
      "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py:24-27:
CurationStatus enum ? STAGING, CURATED, REJECTED; used for video-level workflow state only, no
segment-level maturity.",
      "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py:12-22: LicenseType
enum ? includes Proprietary (self-rated), MIT/Apache/BSD/CC0/CC-BY (vendor-delivered), CC-BY-
NC/CC-BY-NC-SA (non-commercial), PUBLIC_DOMAIN, UNKNOWN; no source tracking (self-rated vs
vendor-delivered not distinguished).",
      "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py:109-122: import_local
sets all local imports to CURATED status by default (line 117: 'Custom local imports bypass
staging by default'), skipping vendor staging workflow.",
      "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py:73-78: License
commercial-safety check via validator.check_license_commercial(license_type) ? whitelist
includes Proprietary (line 21 in validator.py), so self-rated Proprietary is treated as
commercial-safe.",
```

```
      "C:/Users/avidu/Projects/sports-data-collector/src/reporting/adapter.py:113-156:
build_import_recorder() ? observability envelope for import-local; parse stage logs
records_parsed; validation stage reports is_valid (boolean); registration stage captures sport,
license (value string), is_commercial, status (value string); no annotation
author/timestamp/maturity.",
      "C:/Users/avidu/Projects/sports-data-collector/src/reporting/spec.yaml:25-47: Spec rules
emit two flags at import: license_noncommercial (warn if non-commercial), missing_fps (warn if
fps null). Rules only check video-level fields; no segment-level flags for imports.",
      "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:210-237:
insert_badminton_rallies() ? clears existing rallies for video_id then bulk-inserts new ones; no
versioning, audit trail, or per-segment metadata beyond sport schema (rally_number, start_time,
end_time, score, shots_count, ending_reason, last_shot_outcome).",
      "C:/Users/avidu/Projects/sports-data-collector/src/reporting/__init__.py:48-77:
emit_import_report() ? called from curate.py:145-154 after DB insert; optional (soft
dependency); writes next to video or to fallback_dir if video absent; always succeeds (never
raises).",
      "C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py: Issue class
? severity (blocker/high/medium/low), code, message, optional rally_number; only delivery
validation emits Issue objects; import-local does not detect or report Issues.",
      "C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py:29-144:
validate_*_rallies() ? checks temporal non-overlap and positive duration; no quality metrics,
annotator tracking, or maturity scores.",
      "C:/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py:83-112: import
observability envelope captures only is_valid (boolean), errors (list of strings), count
(segments), sport, source_format; no flags for import-local validation issues."
    ],
    "contracts": [
      "VideoMetadata schema: video_id, sport(enum), video_url?, local_path?, license_type(enum),
license_url?, is_commercial(bool), status(CurationStatus enum: staging/curated/rejected),
match_name, fps?, resolution?",
      "CurationStatus enum: STAGING | CURATED | REJECTED ? video-level only, no segment maturity
states",
      "LicenseType enum: MIT, Apache-2.0, BSD, CC0, CC-BY, CC-BY-NC, CC-BY-NC-SA, Public-Domain,
Proprietary, Unknown ? no source marker (self-rated vs vendor-delivered)",
      "is_commercial: boolean derived from license_type whitelist check; whitelist in
config.yaml includes Proprietary (defaults to [MIT, Apache-2.0, BSD, CC0, CC-BY, Public-Domain,
Proprietary])",
      "Import observability envelope (obsreport.RunRecorder): stages=(parse, validation,
registration); parse.metrics.records_parsed; validation.status(ok/error),
validation.metrics.valid(bool); registration.metrics.segments_registered(int),
registration.details={sport, license(str), is_commercial(bool), status(str)}",
      "Delivery observability envelope: stages=(parse, validation);
parse.metrics.rallies_parsed; validation.metrics.issues_blocker/high/medium/low; flags carry
code, severity(error/warn/info), message, stage, customer_message?, evidence={rally_number?,
rally_frames?, fps?}",
      "Spec rules: license_noncommercial (warn if is_commercial==false), missing_fps (warn if
fps==null); rules trigger only at import/delivery validation, never at re-tagging",
      "RallyRecord (CVAT adapter): rally_number, start_time, end_time, shots_count?,
ending_reason?, last_shot_outcome?, score_before?, score_after?, notes?, frame_min, frame_max,
n_boxes, vendor_start_time?, vendor_end_time?",
      "Database cascade: video DELETE triggers cascade on child tables (badminton_rallies,
tennis_points, cricket_deliveries, pickleball_rallies, tabletennis_rallies); child tables have
no independent delete cascade"
    ],
    "gaps_or_mismatches": [
      "No annotation source tracking: LicenseType enum includes Proprietary but does not
distinguish self-rated (manual local import) from vendor-delivered (CVAT/external parser);
license provenance assumes external source unless registered via import-local, but no explicit
marker.",
      "No segment-level maturity: segments (rallies/points/deliveries) lack status/maturity
fields; video-level STAGING/CURATED/REJECTED does not map to individual segment review state
(needs_review -> confirmed -> gold not trackable).",
      "No annotator/author metadata: neither VideoMetadata nor rally schemas capture who
created/reviewed annotations; no timestamp for import or re-tagging.",
      "Local import skips staging workflow: import_local (line 117 in curate.py) hardcodes
```

```
CURATED status, bypassing vendor review pipeline; observability report does not flag this
deviation.",
      "Import validation does not emit observability flags:
DatasetValidator.validate_*_rallies() only check temporal integrity (overlaps, duration);
emit_import_report() logs errors only as string list, not structured Flags; no per-segment QA
metrics.",
      "No re-tagging audit trail: insert_badminton_rallies() (line 215-216 in db.py) DELETE-
then-INSERT erases history when re-importing annotations for the same video_id; no versioning or
conflict detection.",
      "No label quality regression tracking: no metrics for comparing old vs. new annotations
when re-tagging (e.g., shot_count changes, ending_reason divergence); no drift detection.",
      "License whitelist hardcoded in config fallback: check_license_commercial()
(validator.py:21) defaults to fixed list including Proprietary; no per-video or per-sport
override; no rationale/audit trail for why Proprietary is commercial-safe.",
      "Observability rules only video-level: spec.yaml:25-47 defines only video-scope rules
(license_noncommercial, missing_fps); no rules for segment defects at import time (gaps,
outliers, schema violations).",
      "No import-local annotation Issues: CVAT delivery parser emits Issue objects
(blocker/high/medium/low) but import-local does not detect or report Issues to observability
envelope; local vs. vendor validation asymmetry."
    ],
    "files": [
      "C:/Users/avidu/Projects/sports-data-collector/src/core/db.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/cli/curate.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/cli/ingest.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/reporting/__init__.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/reporting/adapter.py",
      "C:/Users/avidu/Projects/sports-data-collector/src/reporting/spec.yaml",
      "C:/Users/avidu/Projects/sports-data-collector/src/parsers/cvat/rally_cvat.py",
      "C:/Users/avidu/Projects/sports-data-collector/tests/test_reporting.py",
      "C:/Users/avidu/Projects/sports-data-collector/tests/test_validator.py",
      "C:/Users/avidu/Projects/sports-data-collector/tests/test_local_import.py"
    ]
  }
]
```

2. assistant (2026-06-08T08:45:44.199Z)

I'll analyze the proposals and findings, verifying key claims against the actual code before issuing my decision. Let me check the load-bearing facts that the proposals disagree on.

3. user (2026-06-08T08:45:45.180Z)

```
52          def import_local(self, args):
53              """Manually registers a local video and annotation file into the database."""
54              sport_str = args.sport.lower()
55              if sport_str not in self.config.get("supported_sports", ["badminton"]):
56                  print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
57                  sys.exit(1)
58
59              # Validate video path
60              if not os.path.exists(args.video_path):
61                  print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")
62
63              # Validate annotation file
64              if not os.path.exists(args.annotations_path):
65                  print(f"Error: Annotations file '{args.annotations_path}' not found.")
66                  sys.exit(1)
67
68              match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
69              video_id = args.video_id or match_name.lower().replace(" ", "_")
```

```
70
71              # Parse license
72              try:
73                  license_type = LicenseType(args.license)
74              except ValueError:
75                  print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")
76                  license_type = LicenseType.UNKNOWN
77
78              is_commercial = self.validator.check_license_commercial(license_type)
79
80              # 1. Parse Annotations
81              annotations = []
82              try:
83                  annotations = self._parse_local_annotations(sport_str, video_id,
args.annotations_path)
84              except Exception as e:
85                  print(f"Error parsing annotations: {e}")
86                  sys.exit(1)
87
88              # 2. Run Validation
89              is_valid = True
90              errors = []
91              if sport_str == "badminton":
92                  is_valid, errors = self.validator.validate_badminton_rallies(annotations)
93              elif sport_str == "tennis":
94                  is_valid, errors = self.validator.validate_tennis_points(annotations)
95              elif sport_str == "cricket":
96                  is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
97              elif sport_str == "pickleball":
98                  is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
99              elif sport_str == "tabletennis":
100                 is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
101
102             if not is_valid:
103                 print(f"Error: Annotations failed validation!")
104                 for err in errors:
105                     print(f"  - {err}")
106                 sys.exit(1)
107
108             # 3. Create Video Metadata
109             video_meta = VideoMetadata(
110                 video_id=video_id,
111                 sport=SportType(sport_str),
112                 video_url=None,
113                 local_path=args.video_path,
114                 license_type=license_type,
115                 license_url=None,
116                 is_commercial=is_commercial,
117                 status=CurationStatus.CURATED,  # Custom local imports bypass staging by
default
118                 match_name=match_name,
119                 fps=args.fps,
120                 resolution=args.resolution
121             )
122
123             # 4. Insert into Database
124             self.db.insert_video(video_meta)
125
126             count = 0
127             if sport_str == "badminton":
128                 count = self.db.insert_badminton_rallies(annotations)
129             elif sport_str == "tennis":
130                 count = self.db.insert_tennis_points(annotations)
131             elif sport_str == "cricket":
```

```
                count = self.db.insert_cricket_deliveries(annotations)
            elif sport_str == "pickleball":
                count = self.db.insert_pickleball_rallies(annotations)
            elif sport_str == "tabletennis":
                count = self.db.insert_tabletennis_rallies(annotations)

            print(f"\nSuccessfully imported local video '{match_name}':")
            print(f"  - Video ID: {video_id}")
            print(f"  - Sport: {sport_str}")
            print(f"  - Commercial Use Allowed: {is_commercial} (License:
{license_type.value})")
            print(f"  - Registered {count} annotated segments.\n")

            # Per-video observability report (next to the video). Never raises.
            from src.reporting import emit_import_report
            src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or
"unknown"
            report_paths = emit_import_report(
                video_meta_obj=video_meta, annotations=annotations, is_valid=True,
                errors=[], count=count, sport=sport_str, source_format=src_fmt,
                fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
            )
            if report_paths:
                print(f"  - Observability report: {report_paths.get('technical_md')}")
                print(f"  - Customer summary    : {report_paths.get('customer_md')}\n")

        def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:
            """Parses simple JSON or CSV annotations files."""
            parsed_items = []
            ext = os.path.splitext(filepath)[1].lower()

            if ext == '.json':
                with open(filepath, 'r', encoding='utf-8') as f:
                    data = json.load(f)

                if not isinstance(data, list):
                    raise ValueError("JSON file must contain a list of objects.")

                # Uses the same safe casts as the CSV path: a missing/blank required
                # time raises a clear ValueError (caught by the caller) instead of a
                # bare KeyError, and optional ints degrade to sensible defaults.
                for idx, obj in enumerate(data):
                    if sport == "badminton":
                        parsed_items.append(BadmintonRally(
                            video_id=video_id,
                            rally_number=_safe_int(obj.get("rally_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            score_before=obj.get("score_before"),
                            score_after=obj.get("score_after"),
                            shots_count=_safe_int(obj.get("shots_count"), None),
                            ending_reason=obj.get("ending_reason"),
                            last_shot_outcome=obj.get("last_shot_outcome")
                        ))
                    elif sport == "tennis":
                        parsed_items.append(TennisPoint(
                            video_id=video_id,
                            point_number=_safe_int(obj.get("point_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            set_score=obj.get("set_score"),
                            game_score=obj.get("game_score"),
                            server=obj.get("server"),
                            ending_type=obj.get("ending_type")
```

```
                    ))
                elif sport == "cricket":
                    parsed_items.append(CricketDelivery(
                        video_id=video_id,
                        over=_safe_int(obj.get("over"), 0),
                        ball=_safe_int(obj.get("ball"), 0),
                        start_time=_safe_float(obj.get("start_time")),
                        end_time=_safe_float(obj.get("end_time")),
                        batsman=obj.get("batsman"),
                        bowler=obj.get("bowler"),
                        runs=_safe_int(obj.get("runs"), None),
                        wicket=bool(obj.get("wicket", False)),
                        extras=_safe_int(obj.get("extras"), 0)
                    ))
                elif sport == "pickleball":
                    parsed_items.append(PickleballRally(
                        video_id=video_id,
                        rally_number=_safe_int(obj.get("rally_number"), idx + 1),
```

4. user (2026-06-08T08:45:45.820Z)

```
        def export_eval_set(self, args):
            """Export curated annotations as indexer-ready eval/training sets.

            Emits, per qualifying video, a `<out_dir>/<sport>/<video_id>.csv` in the
            indexer's `start,end` ground-truth format (decimal seconds), plus a single
            `<out_dir>/manifest.json` that pairs each CSV with its video file. The
            manifest is the contract the downstream eval/training driver reads: the
            indexer keys videos by *file MD5*, not by our `video_id`, so it must run
            against the exact `local_path`/`video_url` recorded here.

            By default only CURATED + commercially-safe videos are exported, mirroring
            the license gate already enforced elsewhere in the toolchain.
            """
            out_dir = args.out_dir
            commercial_only = not args.include_noncommercial
            statuses = [CurationStatus.CURATED]
            if args.include_staging:
                statuses.append(CurationStatus.STAGING)

            supported = self.config.get("supported_sports", ["badminton"])
            sport_strs = [args.sport] if args.sport else supported

            manifest = []
            skipped_noncomm = 0
            skipped_empty = 0

            for sport_str in sport_strs:
                try:
                    sport = SportType(sport_str.lower())
                except ValueError:
                    print(f"Warning: skipping unknown sport '{sport_str}'.")
                    continue

                videos = []
                for status in statuses:
                    videos.extend(self.db.get_videos_by_status(status, sport))

                for video in videos:
                    if commercial_only and not video.is_commercial:
                        skipped_noncomm += 1
                        continue

                    intervals = self.db.get_segment_intervals(video.video_id, sport)
                    if not intervals:
```

```
                        skipped_empty += 1
                        continue

                    sport_dir = os.path.join(out_dir, sport.value)
                    os.makedirs(sport_dir, exist_ok=True)
                    csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
                    with open(csv_path, "w", newline="", encoding="utf-8") as f:
                        writer = csv.writer(f)
                        writer.writerow(["start", "end"])
                        for start, end in intervals:
                            writer.writerow([start, end])

                    manifest.append({
                        "video_id": video.video_id,
                        "sport": sport.value,
                        "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
                        "local_path": video.local_path,
                        "video_url": video.video_url,
                        "match_name": video.match_name,
                        "fps": video.fps,
                        "resolution": video.resolution,
                        "license_type": video.license_type.value,
                        "is_commercial": video.is_commercial,
                        "status": video.status.value,
                        "num_segments": len(intervals),
                    })

            os.makedirs(out_dir, exist_ok=True)
            manifest_doc = {
                "commercial_only": commercial_only,
                "statuses": [s.value for s in statuses],
                "total_videos": len(manifest),
                "total_segments": sum(m["num_segments"] for m in manifest),
                "eval_sets": manifest,
            }
            manifest_path = os.path.join(out_dir, "manifest.json")
            with open(manifest_path, "w", encoding="utf-8") as f:
                json.dump(manifest_doc, f, indent=2)

            print(f"\n=== Exported eval/training set to '{out_dir}' ===")
            if manifest:
                summary = [[m["sport"], m["video_id"], m["num_segments"],
                            "YES" if m["is_commercial"] else "NO",
                            os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]
                           for m in manifest]
                print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
            print(f"\nVideos exported : {len(manifest)}")
            print(f"Total segments  : {manifest_doc['total_segments']}")
            if skipped_noncomm:
                print(f"Skipped (non-commercial) : {skipped_noncomm}  (pass --include-
noncommercial to include)")
            if skipped_empty:
                print(f"Skipped (no annotations) : {skipped_empty}")
            print(f"Manifest        : {manifest_path}\n")

        def convert_cvat(self, args):
            """Convert a vendor CVAT rally export into our canonical per-rally CSV.

            Absorbs the CVAT bounding-box delivery format (rally fields carried as box
            attributes) on the ingestion side: each rally's time is recomputed from
            its frame range at the video's true fps, and content defects (overlaps,
            zero-duration, off-vocabulary endings, coarse sampling) are written to an
            issues report rather than silently "fixed". The emitted CSV imports via
```

```
546             `import-local`; any blocker issue must be resolved first, since the
547             importer's validator rejects overlapping / zero-duration rallies.
548             """
549             from src.parsers.cvat.rally_cvat import (
```

5. assistant (2026-06-08T08:45:49.006Z)

Now let me verify the key claims: the import-local CSV parser behavior, the `_safe_int`/`_safe_float` functions, the validator, the DB insert, and the partial-labels harness.

6. user (2026-06-08T08:45:49.951Z)

```
232             elif ext == '.csv':
233                 with open(filepath, 'r', encoding='utf-8') as f:
234                     reader = csv.DictReader(f)
235                     # Normalize headers
236                     reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if
reader.fieldnames else []
237
238                     for idx, row in enumerate(reader):
239                         if sport == "badminton":
240                             parsed_items.append(BadmintonRally(
241                                 video_id=video_id,
242                                 rally_number=_safe_int(row.get("rally_number"), idx + 1),
243                                 start_time=_safe_float(row.get("start_time")),
244                                 end_time=_safe_float(row.get("end_time")),
245                                 score_before=row.get("score_before"),
246                                 score_after=row.get("score_after"),
247                                 shots_count=_safe_int(row.get("shots_count"), None),
248                                 ending_reason=row.get("ending_reason"),
249                                 last_shot_outcome=row.get("last_shot_outcome")
250                             ))
251                         elif sport == "tennis":
252                             parsed_items.append(TennisPoint(
253                                 video_id=video_id,
254                                 point_number=_safe_int(row.get("point_number"), idx + 1),
255                                 start_time=_safe_float(row.get("start_time")),
256                                 end_time=_safe_float(row.get("end_time")),
257                                 set_score=row.get("set_score"),
258                                 game_score=row.get("game_score"),
259                                 server=row.get("server"),
260                                 ending_type=row.get("ending_type")
261                             ))
262                         elif sport == "cricket":
263                             parsed_items.append(CricketDelivery(
264                                 video_id=video_id,
265                                 over=_safe_int(row.get("over"), 0),
266                                 ball=_safe_int(row.get("ball"), 0),
267                                 start_time=_safe_float(row.get("start_time")),
268                                 end_time=_safe_float(row.get("end_time")),
269                                 batsman=row.get("batsman"),
270                                 bowler=row.get("bowler"),
271                                 runs=_safe_int(row.get("runs"), None),
272                                 wicket=(row.get("wicket") or "").lower() in ["true", "1",
"yes"],
273                                 extras=_safe_int(row.get("extras"), 0)
274                             ))
275                         elif sport == "pickleball":
276                             parsed_items.append(PickleballRally(
277                                 video_id=video_id,
278                                 rally_number=_safe_int(row.get("rally_number"), idx + 1),
279                                 start_time=_safe_float(row.get("start_time")),
280                                 end_time=_safe_float(row.get("end_time")),
281                                 score_before=row.get("score_before"),
```

```
                                score_after=row.get("score_after"),
                                shots_count=_safe_int(row.get("shots_count"), None),
                                ending_reason=row.get("ending_reason"),
                                last_shot_outcome=row.get("last_shot_outcome")
                            ))
                        elif sport == "tabletennis":
                            parsed_items.append(TableTennisRally(
                                video_id=video_id,
                                rally_number=_safe_int(row.get("rally_number"), idx + 1),
                                start_time=_safe_float(row.get("start_time")),
                                end_time=_safe_float(row.get("end_time")),
                                score_before=row.get("score_before"),
                                score_after=row.get("score_after"),
                                shots_count=_safe_int(row.get("shots_count"), None),
                                ending_reason=row.get("ending_reason"),
                                last_shot_outcome=row.get("last_shot_outcome")
                            ))
            else:
                raise ValueError("Unsupported annotations file extension. Use .json or
.csv.")
```

7. user (2026-06-08T08:45:50.787Z)

```
"""Shared parsing helpers used by the CLI importer and dataset parsers.

Previously these were duplicated as private ``_safe_int`` / ``_safe_float`` /
``_parse_time_string`` in both ``src/cli/curate.py`` and the CoachAI parser.
"""
from typing import Any, Optional


def safe_int(value: Any, default: Optional[int] = 0) -> Optional[int]:
    """Parse an int from a possibly-empty/float/None value; return default on
failure."""
    if value is None:
        return default
    s = str(value).strip()
    if not s:
        return default
    try:
        return int(float(s))
    except (TypeError, ValueError):
        return default


def safe_float_required(value: Any) -> float:
    """Parse a float; raise ValueError if missing/empty (caller treats it as
required)."""
    if value is None:
        raise ValueError("missing required numeric value")
    s = str(value).strip()
    if not s:
        raise ValueError("missing required numeric value")
    return float(s)


def parse_time_string(time_str: Any) -> float:
    """Convert 'HH:MM:SS' / 'MM:SS' / float-strings to float seconds; 0.0 if
blank/invalid."""
    if not time_str:
        return 0.0
    s = str(time_str).strip()
    if ":" in s:
        parts = s.split(":")
```

```
            try:
                if len(parts) == 3:
                    return float(parts[0]) * 3600 + float(parts[1]) * 60 + float(parts[2])
                if len(parts) == 2:
                    return float(parts[0]) * 60 + float(parts[1])
            except ValueError:
                return 0.0
        try:
            return float(s)
        except ValueError:
            return 0.0

```

8. user (2026-06-08T08:45:51.232Z)

```
    from typing import List, Tuple, Union
    from src.core.config import load_config
    from src.core.schemas.base import VideoMetadata, LicenseType
    from src.core.schemas.badminton import BadmintonRally
    from src.core.schemas.tennis import TennisPoint
    from src.core.schemas.cricket import CricketDelivery
    from src.core.schemas.pickleball import PickleballRally
    from src.core.schemas.tabletennis import TableTennisRally

    class DatasetValidator:
        def __init__(self, config_path: str = "config/config.yaml"):
            self.config_path = config_path
            self.commercial_licenses = self._load_licenses_whitelist()

        def _load_licenses_whitelist(self) -> List[str]:
            # load_config returns {} on a missing/unreadable file, so the default list
            # below is the single fallback (keep in sync with config.yaml).
            config = load_config(self.config_path)
            return config.get(
                'commercial_licenses',
                ["MIT", "Apache-2.0", "BSD", "CC0", "CC-BY", "Public-Domain",
"Proprietary"],
            )

        def check_license_commercial(self, license_type: Union[LicenseType, str]) -> bool:
            """Determines if a license type is safe for commercial use based on the
configuration whitelist."""
            val = license_type.value if isinstance(license_type, LicenseType) else
license_type
            return val in self.commercial_licenses

        def validate_badminton_rallies(self, rallies: List[BadmintonRally]) -> Tuple[bool,
List[str]]:
            """
            Validates badminton rallies:
            - Sorted chronologically.
            - Start time < End time (handled by Pydantic, but double checked).
            - No temporal overlap between sequential rallies.
            """
            errors = []
            if not rallies:
                return True, errors

            # Sort by rally number to check progression
            sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)

            for i in range(len(sorted_rallies)):
                r = sorted_rallies[i]
                if r.start_time >= r.end_time:
                    errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
```

```
end_time ({r.end_time})")
47
48                  if i > 0:
49                      prev = sorted_rallies[i - 1]
50                      if r.start_time < prev.end_time:
51                          errors.append(
52                              f"Rally overlap detected between Rally {prev.rally_number} "
53                              f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
54                              f"(starts at {r.start_time}s)"
55                          )
56
57              return len(errors) == 0, errors
58
59          def validate_tennis_points(self, points: List[TennisPoint]) -> Tuple[bool,
List[str]]:
60              errors = []
```

9. assistant (2026-06-08T08:45:54.271Z)

I notice the import-local CSV path uses `_safe_float` (not `safe_float_required`) for start_time/end_time. The pragmatist proposal claims `safe_float_required` is used. Let me verify which alias `_safe_float` resolves to in curate.py, and check the DB insert and the partial-labels harness.

10. user (2026-06-08T08:45:56.326Z)

```
11:from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
175:                        rally_number=_safe_int(obj.get("rally_number"), idx + 1),
176:                        start_time=_safe_float(obj.get("start_time")),
177:                        end_time=_safe_float(obj.get("end_time")),
180:                        shots_count=_safe_int(obj.get("shots_count"), None),
187:                        point_number=_safe_int(obj.get("point_number"), idx + 1),
188:                        start_time=_safe_float(obj.get("start_time")),
189:                        end_time=_safe_float(obj.get("end_time")),
198:                        over=_safe_int(obj.get("over"), 0),
199:                        ball=_safe_int(obj.get("ball"), 0),
200:                        start_time=_safe_float(obj.get("start_time")),
201:                        end_time=_safe_float(obj.get("end_time")),
204:                        runs=_safe_int(obj.get("runs"), None),
206:                        extras=_safe_int(obj.get("extras"), 0)
211:                        rally_number=_safe_int(obj.get("rally_number"), idx + 1),
212:                        start_time=_safe_float(obj.get("start_time")),
213:                        end_time=_safe_float(obj.get("end_time")),
216:                        shots_count=_safe_int(obj.get("shots_count"), None),
223:                        rally_number=_safe_int(obj.get("rally_number"), idx + 1),
224:                        start_time=_safe_float(obj.get("start_time")),

[Showing results with pagination = limit: 20]
```

11. user (2026-06-08T08:45:57.103Z)

```
210         def insert_badminton_rallies(self, rallies: List[BadmintonRally]) -> int:
211             if not rallies:
212                 return 0
213             with self._get_connection() as conn:
214                 cursor = conn.cursor()
215                 # First clear existing rallies for this video_id to avoid key conflicts on
updates
216                 video_id = rallies[0].video_id
217                 cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?",
(video_id,))
218
219                 for rally in rallies:
220                     cursor.execute("""
```

```
221                     INSERT INTO badminton_rallies (
222                         video_id, rally_number, start_time, end_time,
223                         score_before, score_after, shots_count, ending_reason,
last_shot_outcome
224                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
225                     """, (
226                         rally.video_id,
227                         rally.rally_number,
228                         rally.start_time,
229                         rally.end_time,
230                         rally.score_before,
231                         rally.score_after,
232                         rally.shots_count,
233                         rally.ending_reason,
234                         rally.last_shot_outcome
235                     ))
236                 conn.commit()
237                 return len(rallies)
238 
239         def insert_tennis_points(self, points: List[TennisPoint]) -> int:
240             if not points:
241                 return 0
242             with self._get_connection() as conn:
243                 cursor = conn.cursor()
244                 video_id = points[0].video_id
```

12. user (2026-06-08T08:45:57.737Z)

```
40      Config = Tuple[float, float, float]
41 
42      # Operating points to compare. The tuned config wins on its fit-video but over-segments
43      # and does not transfer; baseline (+ net-crossing gate) is the stronger general point.
44      TUNED: Config = (0.05, 1.0, 1.0)
45 
46 
47      def restrict_to_window(intervals: List[Interval], w0: float, w1: float) ->
List[Interval]:
48          """Keep intervals that overlap the labeled window [w0, w1].
49 
50          A predicted segment is evaluated iff it overlaps the labeled region; segments wholly
51          in the un-labeled tail (start >= w1) or before the window (end <= w0) are dropped so
52          they are neither rewarded nor penalized. GTs are assumed already inside the window.
53          """
54          return [(s, e) for (s, e) in intervals if e > w0 and s < w1]
55 
56 
57      def _eval_point(predict_fn, cfg: Config, points, fps: float, gts: List[Interval],
58                      window: Tuple[float, float], gate_crossings: int, iou: float) -> dict:
59          """Score one (config, gate) operating point inside the labeled window."""
60          preds = predict_fn(cfg)
61          if gate_crossings > 0 and points:
62              preds = rally_gate.apply_gate(points, preds, fps, min_crossings=gate_crossings)
63          preds = restrict_to_window(preds, *window)
64          sc = score(preds, gts, iou)
65          return {"cfg": cfg, "gate": gate_crossings, "n_pred": len(preds), "score": sc,
"preds": preds}
66 
67 
68      def run(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
69              window_start: float = 0.0, window_end: Optional[float] = None,
70              gate_crossings: int = 2, iou: float = 0.5,
71              comparison_out: Optional[str] = None) -> dict:
72          gts_all = load_golden_gts(labels_csv)
73          if not gts_all:
74              raise SystemExit(f"no usable human rallies in {labels_csv}")
```

```
75          if window_end is None:
76              window_end = max(e for _, e in gts_all)
77          window = (window_start, window_end)
78          gts = restrict_to_window(gts_all, *window)
79
80          predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
81          points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
82          traj_end = (max(p.frame for p in points) / fps) if points else 0.0
83
84          print("=== Pipeline vs PARTIAL human labels ===")
```

13. assistant (2026-06-08T08:46:04.213Z)

Confirmed a critical correction: in curate.py line 11, `_safe_float` is aliased to `safe_float_required`, so the CSV path DOES use the required (raises on missing) variant ? the pragmatist's claim is correct, and one blank `end_time` aborts the whole import. Also confirmed `_safe_int` does `int(float(s))` so fractional rally numbers truncate silently, and the DB insert is DELETE-then-INSERT (last-write-wins, no audit). The partial-labels harness defaults `window_end` to `max(end_time)`, confirming the pragmatist's "recoverable from exported CSV" claim and the lifecycle proposal's "wrong when labeled past last rally" caveat.

Let me verify the two source CSV artifacts (lean vs rich) and the rich format's actual columns, since the P0 scope decision hinges on which is the source of truth.

14. user (2026-06-08T08:46:05.177Z)

```
1       rally_number,start_time,end_time,ending_reason,sport
2       1,37.434,72.194,winner,badminton
3       2,79.936,112.954,winner,badminton
4       3,130.193,134.947,winner,badminton
5       4,139.697,144.944,unforced_error,badminton
6       5,152.196,157.198,unforced_error,badminton
```

15. user (2026-06-08T08:46:06.127Z)

```
1       rally,start_mmss,end_mmss,dur_s,shots,ending_reason,ending_consistent,notes,auto_flag,VE
RIFIED(Y/N),true_start_mmss,true_end_mmss,true_ending,reviewer_comment
2       1,0:03.003,0:21.021,18.0,15,other,yes,net,,,,,,
3       2,0:02.502,0:32.532,30.0,2,forced_error,yes,,OVERLAP;INFLATED?,,,,,
4       3,0:44.544,0:51.051,6.5,5,unforced_error,yes,,,,,,,
```

16. user (2026-06-08T08:46:06.991Z)

```
424     def load_canonical_csv(csv_path: str) -> List[RallyRecord]:
425         """Load a canonical per-rally CSV into RallyRecords, so a *CSV* delivery (not
426         just a CVAT export) can be validated. Decimal start/end preferred, mm:ss
427         fallback; rows with no usable start/end are skipped. (frame_* are -1 here, as a
428         CSV carries no frame info ? only the frame-derived checks become inapplicable.)
429         """
430         import csv as _csv
431         if not os.path.exists(csv_path):
432             raise FileNotFoundError(f"CSV not found: {csv_path}")
433         records: List[RallyRecord] = []
434         with open(csv_path, newline="", encoding="utf-8") as f:
435             reader = _csv.DictReader(f)
436             reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
[])]
437             for raw in reader:
438                 row = {(k or "").strip().lower(): v for k, v in raw.items()}
439                 rn = safe_int(row.get("rally_number"), None)
440                 if rn is None:
441                     continue
442                 start = _parse_secs(row.get("start_time"))
443                 if start is None:
```

```
444                     start = _parse_secs(row.get("start_mmss"))
445                 end = _parse_secs(row.get("end_time"))
446                 if end is None:
447                     end = _parse_secs(row.get("end_mmss"))
448                 if start is None or end is None:
449                     continue
450                 records.append(RallyRecord(
451                     rally_number=rn, start_time=start, end_time=end,
452                     shots_count=safe_int(row.get("shots_count"), None),
453                     ending_reason=(row.get("ending_reason") or "").strip() or None,
454                     last_shot_outcome=(row.get("last_shot_outcome") or "").strip() or None,
455                     score_before=(row.get("score_before") or "").strip() or None,
456                     score_after=(row.get("score_after") or "").strip() or None,
457                     notes=(row.get("notes") or "").strip() or None,
458                     frame_min=-1, frame_max=-1, n_boxes=0,
459                 ))
460         return records
461
462
463     def build_summary(records: List[RallyRecord], issues: List[Issue],
```

17. assistant (2026-06-08T08:46:11.836Z)

All key claims are verified. Important nuance confirmed: `load_canonical_csv` keys on `rally_number` (line 439) but the rich `rally_review.csv` uses a `rally` column (not `rally_number`), so even the canonical CSV reader would skip every row ? the canonical-hub proposal's claim that it "ALREADY handles mm:ss" is true for the helper but it would not pick up the `rally` column without an alias. This strengthens routing the rich file through an offline converter rather than the canonical reader.

I have everything I need to make the decision. Let me also confirm the `videos` table schema columns and CurationStatus to ground the lifecycle recommendations.

18. user (2026-06-08T08:46:13.073Z)

```
1       from pydantic import BaseModel, Field
2       from typing import Optional
3       from enum import Enum
4
5       class SportType(str, Enum):
6           BADMINTON = "badminton"
7           TENNIS = "tennis"
8           TABLE_TENNIS = "tabletennis"
9           PICKLEBALL = "pickleball"
10          CRICKET = "cricket"
11
12      class LicenseType(str, Enum):
13          MIT = "MIT"
14          APACHE_2 = "Apache-2.0"
15          BSD = "BSD"
16          CC0 = "CC0"
17          CC_BY = "CC-BY"
18          CC_BY_NC = "CC-BY-NC"
19          CC_BY_NC_SA = "CC-BY-NC-SA"
20          PUBLIC_DOMAIN = "Public-Domain"
21          PROPRIETARY = "Proprietary"
22          UNKNOWN = "Unknown"
23
24      class CurationStatus(str, Enum):
25          STAGING = "staging"
26          CURATED = "curated"
27          REJECTED = "rejected"
28
29      class VideoMetadata(BaseModel):
30          video_id: str = Field(..., description="Unique alphanumeric identifier for the
```

```
video")
31          sport: SportType
32          video_url: Optional[str] = Field(None, description="Remote URL of the video (if
available)")
33          local_path: Optional[str] = Field(None, description="Local path to video file (if
downloaded or custom)")
34          license_type: LicenseType
35          license_url: Optional[str] = None
36          is_commercial: bool = Field(False, description="Flag indicating if the license
allows commercial use")
37          status: CurationStatus = CurationStatus.STAGING
38          match_name: str = Field(..., description="Name of the match/event")
39          fps: Optional[float] = Field(None, description="Frames per second")
40          resolution: Optional[str] = Field(None, description="Resolution string (e.g. 1080p,
720p)")
41
```

19. assistant (2026-06-08T08:46:20.824Z)

All claims are grounded. I have verified the load-bearing facts and am ready to issue the merged decision. Key verified facts driving my calls:

- The lean `Adarsh and Avi.rallies.csv` (header `rally_number,start_time,end_time,ending_reason,sport`, integer rallies, decimal seconds, complete pairs) matches the import-local DictReader exactly ? P0 ships today with zero code changes.
- `_safe_float` aliases `safe_float_required` (curate.py:11), so one blank `end_time` aborts the whole import ? a real partial-label landmine.
- `_safe_int` does `int(float(s))` (parsing.py:17), silently truncating fractional rally numbers; DB insert is DELETE-then-INSERT (db.py:217), last-write-wins, no audit.
- `restrict_to_window` defaults `window_end=max(end_time)` (eval_partial_labels.py:75-76) ? recoverable from the exported CSV, but wrong when labeled past the last rally.
- The rich `rally_review.csv` uses `rally` (not `rally_number`), mm:ss times, pipe-delimited reasons ? neither import-local nor `load_canonical_csv` (which keys on `rally_number`) ingest it; offline conversion is required.

20. user (2026-06-08T08:47:55.480Z)

Structured output provided successfully

21. assistant (2026-06-08T08:48:06.604Z)

I've issued the merged authoritative recommendation. Summary of the key calls:

Positioning: sports-data-collector is the canonical system-of-record for golden rally labels, with the indexer consuming only the intervals-only export contract. End-state is a full label lifecycle (provenance + maturity + labeled-window in the manifest); the pragmatic P0 ships partial-label ingestion this week.

P0 (ship now): Import the lean `Adarsh and Avi.rallies.csv` via unmodified import-local (verified header-exact match to curate.py:232-250) + a thin pre-import normalizer that drops blank/bad-end rows (which otherwise abort the whole import via `safe_float_required`, curate.py:11) and rejects/renumbers fractional rally_number (which silently truncates and collides on the PK via DELETE-then-INSERT).

P1: CLI `--window-end=max(end_time)` workaround, then persist `labeled_window_start/end`; add the maturity ladder (needs_review?confirmed?gold) + annotation_source/timestamp provenance, routing self-rated imports to needs_review.

P2: Unify sources behind one adapter interface with Issue detection on manual imports; the offline rally_review.csv?lean converter (only if the rich file is the source of truth); freeze and test the export contract.

Conflicts resolved (recorded in disagreements): I sided with the pragmatist on P0 scope

(ship via existing path, not the L-effort adapter framework), kept the rich rally_review.csv out of the importer (it uses a `rally` column that neither import-local nor `load_canonical_csv` ingests), phased the labeled-window from CLI-now to schema-later, deferred needs_review/maturity to P1, fixed maturity at video-level, and ruled out widening the INTEGER rally_number PK since no consumer reads it.

One open question gates the P2 converter: whether the lean file or the rich rally_review.csv is the source of truth for this week's import.